TEC | Tecnológico de Costa Rica

Bases de datos - Segundo Proyecto

Ariel Sevilla Matamoros - 2025075426
Alejandro Zúñiga Martínez - 2025113729

Prof. Jacqueline Solís Céspedes

Segundo semestre 2026

Fecha de entrega: 25/10/2026

Contenido

1. Dictamen de rechazo y propuesta de factibilidad técnica 4
2. Análisis de factibilidad 5
2.1 Factibilidad técnica 5
2.2 Factibilidad económica 5
2.3 Factibilidad operativa 7
2.4 Análisis de riesgos 7
3. Análisis de requisitos y dimensionamiento de datos 8
3.1 Requisitos funcionales 8
3.2 Requisitos no funcionales 8
3.3 Volúmenes de datos 9
4. Diseño de la base de datos 10
4.1 Diseño conceptual 10
4.2 Diseño lógico 14
Diagrama del modelo 14
Tabla de relaciones y cardinalidades 14
Tablas del modelo, bloques A a E 15
Normalización 16
Supuestos de diseño 16
4.3 Diseño físico 17
4.4 Seguridad y roles 17
5. Implementación y pruebas 23
5.1 DDL y restricciones 23
5.2 Transacciones, triggers y procedimientos 23
5.3 Prueba de concurrencia 24
5.4 Consultas analíticas 24
5.5 EXPLAIN ANALYZE 26
6. Operación y mantenimiento 34
6.1 Respaldos. 34
6.2 RPO 34
6.3 RTO 34
6.4 SLA de 99.99%. 34
6.5 Recuperación ante desastres. 34
6.6 Privacidad de datos. 34
7. Interfaz gráfica 34
7.1 Cómo ejecutarla 35
7.2 Cliente 35
7.3 Taquillero y operador de sala 37
7.4 Administrador 37
7.5 Alcance y limitaciones 42
8. Repositorio y evidencias 42
9. Declaración de IA: 43

10. Referencias..... 43

1. Dictamen de rechazo y propuesta de factibilidad técnica

Tras hacer la evaluación de la infraestructura propuesta por el proveedor saliente contra la carga esperada del estreno mundial, está siendo de 8 millones de usuarios concurrentes, con un pico de 50,000 TPS durante 10 minutos, involucrando transacciones ACID, tener que cumplir PCI-DSS y con retención de auditoria de 5 años, el equipo la rechaza. Ninguno de los 6 componentes cumple con los requisitos mínimos del negocio y varios violan directamente las reglas de negocio y normas de pago. A continuación, las discrepancias.

1. Motor de base de datos, Access y SQLite son motores embebidos pensados para uso personal o de pocos usuarios, sin servidor de base de datos propiamente dicho. SQLite permite un solo escritor a la vez, es decir que bloquea la base completa al escribir y Access tiene un tope de tamaño de archivo 2 GB y 255 usuarios simultáneos teóricos (Microsoft, 2026). Frente a 50,000 TPS con bloques y compra de asientos, el sistema se serializaría y colapsaría en segundos. Además, es un único punto de falla: sin réplica, sin alta disponibilidad, sin roles reales (RN-17) y sin capacidad de almacenar varios TB de logs por 5 años (RN-20).
2. Infraestructura y red, 50 Mbps equivalen a 6.25 MB/s. Repartidos entre 8 millones de usuarios concurrentes, tocan unos 6 bits por segundo por usuario. Aun suponiendo solo 2 KB por solicitud, 50,000 TPS exigirían unos 100 MB/s, es decir 800 Mbps, 16 veces la capacidad del enlace, y eso sin contar el Wi-Fi compartido. Un enlace residencial no tiene SLA, ni redundancia, ni protección contra ataques DDoS, ni latencia razonable para cines en otros continentes.
3. Control de asientos en una hoja de cálculo compartida de Google Drive. Una hoja de cálculo no es un sistema transaccional: no tiene ACID, ni restricciones de unicidad, ni bloqueos con expiración. Si dos clientes escriben su nombre en el mismo asiento, gana el último en guardar: la doble venta está garantizada. Tampoco puede implementar el bloqueo temporal de 2 minutos (RN-05), la unicidad del boleto (RN-12) ni el mapa en tiempo real (RN-03), y el nombre escrito a mano no identifica a nadie. Google además limita el tamaño de una hoja a 20 millones de celdas (Google Workspace Updates, 2026; el límite anterior era de 10 millones, según Google, s. f.-a) y las solicitudes por minuto de su API (Google, s. f.-b), muy por debajo de la carga requerida.
4. Seguridad y pagos es la falla más grave. PCI-DSS prohíbe almacenar el código de seguridad después de la autorización (Gray, 2017) y exige proteger el número de tarjeta en tránsito y en reposo, enviarlos en claro a un chat grupal los expone a cualquiera con acceso al grupo. Incumple la RN-09, y el cobro manual por un pasante rompe la atomicidad entre pago y asiento (RN-10 y RN-11). La exposición es tanto legal (multas o demandas) como reputacional.
5. Plan de respaldo “reiniciar y cruzar los dedos”. No existe plan. Sin backups ni réplica, el RPO es toda la información desde la última copia que no existe y el RTO es indefinido. Esto contradice el objetivo de disponibilidad, que es de 99.99%, unos 52 minutos de caída al año como máximo, y pone en riesgo la retención fiscal de 5 años (RN-20).
6. Dimensionamiento
   La propuesta no calcula carga, almacenamiento ni crecimiento. No hay estimación de TPS, ni de volumen de logs de transacciones, bloqueos y accesos, ni presupuesto para retener 5 años de auditoría. Un proyecto de este tamaño no puede aprobarse sin números.

Conclusión: la propuesta legada incumple como mínimo las reglas RN-03, RN-05, RN-09, RN-10, RN-11, RN-12, RN-14, RN-17 y RN-20, y es inviable técnica, económica y legalmente. Se sustituye por la arquitectura propuesta en las siguientes secciones: base de datos relacional administrada con alta disponibilidad multizona y réplica multirregión, cómputo escalable, WAF, pasarela de pagos certificada, almacenamiento escalonado para logs y respaldos automatizados.

2. Análisis de factibilidad

2.1 Factibilidad técnica

La infraestructura legada no aguanta un estreno mundial, como explicamos en el dictamen. Proponemos una arquitectura en la nube con estas partes. En la entrada va una CDN con WAF y protección contra DDoS, más una sala de espera virtual para que los 8 millones de usuarios no lleguen todos al mismo tiempo a la base de datos. La aplicación corre en contenedores que escalan solos según la demanda, con una caché para el mapa de asientos. La base de datos es PostgreSQL administrado, con alta disponibilidad entre zonas y una réplica en otra región. Los pagos pasan por una pasarela externa certificada (en este proyecto se simula). El histórico de 5 años se guarda en almacenamiento de archivo, y lo reciente en almacenamiento rápido.

Elegimos PostgreSQL porque cumple lo que el negocio necesita: transacciones ACID, restricciones de unicidad para evitar la doble venta, particionamiento por fecha para los logs e índices parciales para buscar asientos disponibles.

2.2 Factibilidad económica

Estimamos los costos mensuales con dos proveedores, AWS y Azure. Usamos como base los valores de referencia del enunciado y los comparamos con los precios unitarios que publica cada proveedor.

Almacenamiento a 5 años

Para calcularlo usamos estos supuestos: 2.5 boletos por compra (6 millones de compras al mes), 1.3 intentos de pago por compra, 3 asientos bloqueados y liberados por cada boleto vendido, 5% de reintentos en puerta y 0.5% de boletos con alguna operación administrativa. Calculamos dos escenarios. En el base, las filas miden 500, 200, 150, 600, 400, 300 y 250 bytes (en el orden de la tabla) y se suma 40% por índices. En el conservador miden 2000, 400, 300, 2000, 600, 500 y 500 bytes y se suma 50%, porque los logs de pasarela suelen guardar la respuesta completa. Cada compra tiene un solo pago; los 1.3 intentos por compra, incluidos los rechazados, quedan registrados en transaccion_log.

Tabla	Filas al mes	GB al mes (base)	GB al mes (conservador)
transaccion_log	15.6 M	10.9	46.8
asiento_auditoria_log	120 M	33.6	72.0
acceso_puerta_log	15.75 M	3.3	7.1
admin_operacion_audit	0.075 M	0.06	0.23
compra	6 M	3.4	5.4
boleto	15 M	6.3	11.25
pago	6 M	2.1	4.5
Total mensual		59.7	147.3

Acumulado a 5 años: sin crecimiento son 3.6 TB (base) y 8.8 TB (conservador). Con un crecimiento de 10% anual son 4.4 TB y 10.8 TB. Si en el conservador contamos 3 copias, primaria, réplica y respaldos, llegamos a 32.4 TB. Usamos este último valor para justificar el costo de almacenamiento.

Comparativa de costos mensuales con los valores dados

Componente	AWS	Azure
Base de datos transaccional	Aurora PostgreSQL multirregión con autoescalado: 12,500 USD	Azure Database for PostgreSQL Flexible Server con alta disponibilidad: 11,800 USD
Cómputo / frontend	EKS, 50 pods en pico: 6,200 USD	AKS: 5,900 USD
Almacenamiento histórico y 5 años de logs	S3 Standard + S3 Glacier Flexible Archive: 1,800 USD	Blob Storage + Archive Tier: 1,750 USD
Seguridad y red	CloudFront + WAF + Global Accelerator: 3,500 USD	Front Door + WAF: 3,400 USD
Total mensual (OPEX)	24,000 USD	22,850 USD

Para respaldar esos valores buscamos los precios unitarios publicados:

Servicio	Precio	Fuente
Aurora, almacenamiento	0.10 USD por GB al mes	Amazon Web Services (s. f.-a)
Aurora, operaciones de I/O	0.20 USD por millón	Amazon Web Services (s. f.-a)
Aurora, instancia db.r6i.large	0.29 USD por hora (unos 212 USD al mes)	Amazon Web Services (s. f.-a)
AWS WAF	5 USD por web ACL, 1 USD por regla y 0.60 USD por millón de solicitudes	Amazon Web Services (s. f.-b)
Azure Front Door	Base de 35 USD (Standard) o 330 USD (Premium) al mes, 0.083 USD por GB de salida en Norteamérica (primeros 10 TB), WAF incluido en Premium	Microsoft (s. f.)
S3 Standard	0.023 USD por GB al mes	Cloudian (s. f.)
S3 Glacier Flexible Retrieval / Deep Archive	0.0036 / 0.00099 USD por GB al mes	Cloudian (s. f.)
Azure Blob, Hot / Cool / Archive	0.018 / 0.01 / 0.00099 USD por GB al mes	CloudZero (2026)
Azure PostgreSQL, D4ds_v5 / D8ds_v5	260 / 520 USD al mes solo de cómputo en East US, y la alta disponibilidad lo duplica aproximadamente	Bytebase (s. f.)

Con esos precios, los 32.4 TB de almacenamiento costarían unos 745 USD al mes en S3 Standard, 117 en Glacier Flexible, 583 en Blob Hot y 32 en Blob Archive, así que la línea de 1,750 a 1,800 USD del ejemplo cubre el almacenamiento más solicitudes y replicación. Para el WAF, con 1 ACL, 20 reglas y unos 1,000 millones de solicitudes al mes serían 625 USD, dentro de lo que cuesta la línea de seguridad y red. En la base de datos, un D8ds_v5 con alta disponibilidad ronda los 1,040 USD, así que los 11,800 USD de Azure equivalen a unas 11 instancias de ese tamaño: el ejemplo asume un clúster con réplicas, no un servidor solo.

Costo por millón de transacciones

En Azure son 22,850 USD entre 15 millones de boletos, o sea 0.0015 USD por boleto: unos 1,523 USD por millón de boletos y 3,808 USD por millón de compras. En AWS son 0.0016 USD por boleto: 1,600 USD por millón de boletos y 4,000 USD por millón de compras.

Proyección a 3 años

Usamos un CAPEX inicial de 15,000 USD (consultoría de arquitectura e implementación, igual en ambos proveedores) y un crecimiento del OPEX de 10% por año.

Concepto	Azure	AWS
CAPEX	15,000	15,000
OPEX año 1	274,200	288,000
OPEX año 2	301,620	316,800
OPEX año 3	331,782	348,480
Total a 3 años	922,602	968,280

Retorno de la inversión

La infraestructura cuesta unos 0.0015 USD por boleto procesado y asegurado. Si suponemos un precio promedio de 10 USD por boleto, las ventas serían de unos 150 millones de USD al mes, o unos 208,000 USD por hora en promedio, y bastante más durante un estreno. Los 922,602 USD de 3 años equivalen a unas 4.4 horas de ventas promedio, y una sola caída en un estreno mundial costaría más que eso. La arquitectura legada además traería doble venta, fuga de tarjetas e incumplimiento de PCI-DSS.

Proveedor seleccionado

Elegimos Azure porque sale 1,150 USD más barato al mes (4.8%), unos 45,678 USD menos en 3 años, y ofrece base de datos PostgreSQL administrada con alta disponibilidad, almacenamiento por niveles y WAF. AWS es una alternativa viable, solo que un 5% más cara.

2.3 Factibilidad operativa

La solución necesita un equipo de operación (DevOps y DBA) con monitoreo y soporte durante los estrenos, guías para atender incidentes y pruebas de carga antes de cada estreno grande. También hay que capacitar al personal de taquilla, puerta y contabilidad según su rol. La migración se haría por fases: primero unos cines piloto y después el resto, para no depender de una sola fecha de cambio. Como son servicios administrados, parches, respaldos y failover los maneja el proveedor, y eso baja mucho el trabajo de operación.

2.4 Análisis de riesgos

ID	Descripción del riesgo	Probabilidad	Impacto	Estrategia de mitigación / Plan de respaldo (DRP)	RTO / RPO objetivo
R-01	Caída de la instancia principal de BD durante el estreno	Media	Crítico	Failover automático a una réplica en otra zona y réplica multirregión	RTO < 5 min / RPO ≈ 0
R-02	Caída o rechazo masivo de la pasarela de pago	Media	Crítico	Pasarela secundaria con cambio automático, reintentos con clave de idempotencia y liberación de asientos según RN-10	RTO < 5 min / RPO 0
R-03	Ataque de denegación de servicio (DDoS)	Alta	Crítico	WAF, CDN, límite de solicitudes por IP y sala de espera virtual	RTO < 5 min / RPO 0
R-04	Pérdida de consistencia del inventario	Media	Crítico	Restricción UNIQUE, transacciones ACID, expiración automática de bloqueos y pruebas de concurrencia con 20 a 50 solicitudes simultáneas	Preventivo / RPO 0
R-05	Saturación por el pico de 50,000 TPS	Alta	Crítico	Autoescalado, réplicas de lectura, caché de disponibilidad, colas y pruebas de carga antes del estreno	RTO < 5 min / RPO 0
R-06	Fuga de datos de pago o personales	Baja	Crítico	Pasarela certificada PCI-DSS, sin PAN ni CVV en la BD, TLS, cifrado en reposo, RBAC y auditoría	Preventivo / RPO 0
R-07	Corrupción o borrado de datos, accidental o malicioso	Baja	Crítico	Restauración a un punto en el tiempo (PITR), respaldos inmutables, bitácoras de solo inserción y privilegios mínimos	RTO < 1 h / RPO ≤ 5 min
R-08	Falla de una zona o región completa del proveedor	Baja	Crítico	Despliegue en varias zonas, réplica multirregión y cambio automático de DNS o Front Door	RTO < 15 min / RPO ≤ 5 s
R-09	Fraude en puerta con QR duplicado o falsificado	Media	Medio	QR firmado y único, validación de un solo uso (RN-14) y registro en acceso_puerta_log	Preventivo / RPO 0
R-10	Modificación indebida por un administrador	Baja	Crítico	RBAC y bitácora admin_operacion_audit con fecha, IP, rol y usuario, más revisión periódica	Preventivo / RPO 0
R-11	Crecimiento de los logs hasta saturar el almacenamiento	Media	Medio	Particionamiento por fecha, archivado automático a almacenamiento frío y alertas de capacidad	N/A / RPO 0

3. Análisis de requisitos y dimensionamiento de datos

3.1 Requisitos funcionales

RF-01. Mostrar solo las funciones activas con inicio futuro, con título, sinopsis, clasificación, horarios y ubicación de la sala, y poder filtrarlas por ubicación, fecha y género. Reglas relacionadas: RN-01 y RN-02.

RF-02. Mostrar el mapa de la sala en tiempo real, por zonas, con los estados disponible, ocupado, reservado y bloqueado. Regla relacionada: RN-03.

RF-03. Limitar el máximo de asientos por transacción, con el valor configurable. Regla relacionada: RN-04.

RF-04. Bloquear los asientos elegidos por 2 minutos y liberarlos solos si el tiempo se acaba. Regla relacionada: RN-05.

RF-05. Manejar tarifas por perfil, es decir adulto, niño, tercera edad, estudiante. Aplicar las promociones automáticamente como el 2x1 y calcular el total exacto con los cargos por servicio. Reglas relacionadas: RN-06, RN-07 y RN-08.

RF-06. Procesar el pago simulado por medio de una pasarela, sin guardar datos sensibles como el CVV. Si el pago se rechaza, se cancela la compra y los asientos se liberan de inmediato. Reglas relacionadas: RN-09 y RN-10.

RF-07. Emitir el comprobante con un QR único por función, localidad y asiento, solo cuando el pago ya fue aprobado, y dejarlo disponible en el perfil del cliente. Reglas relacionadas: RN-11, RN-12 y RN-13.

RF-08. Validar el QR en la puerta una sola vez, si se intenta de nuevo, mostrar “Entrada ya utilizada”, revisar que sea de la función y horario correctos y respetar la tolerancia de 30 minutos antes a 15 minutos después del inicio. Reglas relacionadas: RN-14, RN-15 y RN-16.

RF-09. Manejar los roles cliente, taquillero, operador de sala, contador y administrador con los permisos mínimos que indica la matriz del enunciado. Regla relacionada: RN-17.

RF-10. Permitir al administrador gestionar películas, salas, mapas de asientos, funciones sin traslapes de horario y la matriz de precios, sin poder modificar una sala que ya tiene entradas vendidas o reservadas. Regla relacionada: RN-18.

RF-11. Permitir cancelaciones y reembolsos según la política del cine, por ejemplo, hasta 2 horas antes de la función, liberando el asiento. Regla relacionada: RN-19.

RF-12. Llevar cuatro bitácoras de auditoría con retención de 5 años: transaccion_log, asiento_auditoria_log, acceso_puerta_log y admin_operacion_audit. Regla relacionada: RN-20.

RF-13. Ofrecer las consultas analíticas de la gerencia: ocupación y recaudación por función, top 5 de películas de la última semana, métodos de pago en porcentaje y auditoría de operaciones críticas del último mes por administrador. Origen: consultas analíticas mínimas del enunciado.

RF-14. Permitir la conciliación bancaria y los cortes de ingresos diarios, semanales y mensuales. Origen: proceso de gestión financiera y reembolsos.

RF-15. Permitir resolver incidencias de acceso en puerta, como un QR ilegible o un cliente fuera del tiempo de tolerancia. Origen: proceso de control de acceso.

RF-16. Mostrar en el panel de administración las métricas en tiempo real. Origen: panel de administración.

3.2 Requisitos no funcionales

RNF-01. Rendimiento: soportar el pico de 50,000 TPS durante 10 minutos. Los 8 millones de usuarios no llegan todos a la base de datos, porque la CDN, la caché y la sala de espera virtual absorben la mayor parte.

RNF-02. Disponibilidad: 99.99%, que son unos 52 minutos de caída al año como máximo.

RNF-03. Recuperación: RPO de 5 segundos o menos y RTO menor a 5 minutos.

RNF-04. Consistencia: transacciones ACID y una restricción de unicidad por función y asiento que impida la doble venta.

RNF-05. Seguridad: cumplir PCI-DSS sin guardar el número completo de la tarjeta ni el CVV, cifrar los datos en tránsito y en reposo, usar WAF y protección DDoS y dar a cada rol solo los permisos que necesita.

RNF-06. Retención: guardar los logs 5 años, en tablas separadas por tipo, particionadas por fecha y de solo inserción para que no se puedan alterar.

RNF-07. Escalabilidad: que el cómputo crezca según la demanda y que haya réplicas de lectura.

RNF-08. Tiempo de respuesta: proponemos como meta que el 95% de las consultas de disponibilidad de asientos respondan en menos de 200 ms, y lo comprobamos con EXPLAIN ANALYZE.

RNF-09. Privacidad: proteger los datos personales de los clientes y limitar quién puede verlos.

RNF-10. Usabilidad: interfaz funcional en distintos dispositivos.

RNF-11. Operación: monitoreo, alertas y respaldos automáticos.

3.3 Volúmenes de datos

Datos del enunciado: 8,000,000 de usuarios concurrentes, 50,000 TPS de pico durante 10 minutos, 30 millones de transacciones en ese tiempo, 500 títulos activos, 1,500 salas y 15 millones de boletos al mes, con 5 años de retención.

Lo que sale de ahí: 15,000,000 entre los 2,592,000 segundos de un mes dan 5.8 TPS promedio, así que el pico es unas 8,640 veces el promedio. Por eso el sistema se diseña para el pico y no para el promedio. Cada sala vende unos 10,000 boletos al mes, unos 333 al día.

Histórico a 5 años, sin contar crecimiento: 900 millones de boletos, 360 millones de compras y 360 millones de pagos (un pago por compra; los intentos rechazados o reintentados quedan en transaccion_log).

Registros de auditoría: 151.4 millones de registros al mes entre las cuatro bitácoras, 15.6 millones de transaccion_log, 120 millones de asiento_auditoria_log, 15.75 millones de acceso_puerta_log y 0.075 millones de admin_operacion_audit. Son unos 1,817 millones al año y unos 9,085 millones en 5 años. El espacio que ocupan está en la tabla de almacenamiento de la sección 2.2, con los supuestos que usamos.

4. Diseño de la base de datos

4.1 Diseño conceptual

El diseño conceptual está hecho con la notación de Chen. Primero va el DER general, con las entidades, las relaciones y sus cardinalidades, y después un diagrama por bloque con los atributos (la llave primaria va subrayada). Solo mostramos la cardinalidad máxima (1, N, M); lo opcional se ve en el modelo lógico, en las columnas que aceptan NULL. Asiento_Funcion es entidad débil porque depende de la función y del asiento. Auditoria representa las cuatro bitácoras, que se relacionan de forma lógica y sin llave foránea. Respecto al enunciado, Transacción equivale a compra, pago y transaccion_log, y Auditoría equivale a las cuatro bitácoras.

Figura 1
DER conceptual general (notación de Chen)

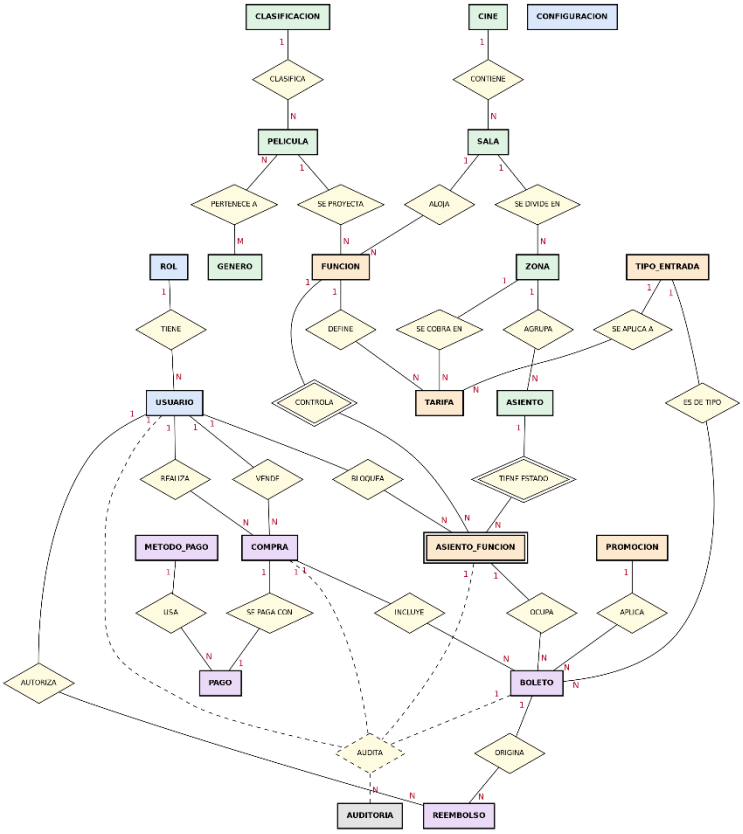


Figura 2
DER conceptual, bloque A: usuarios y configuración

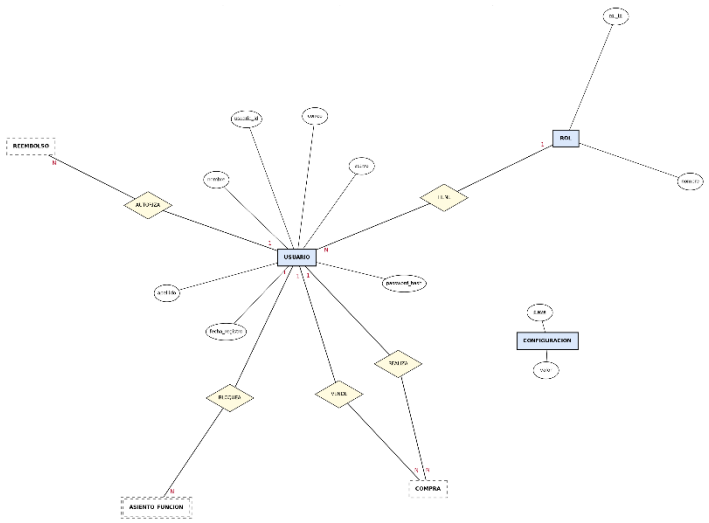


Figura 3
DER conceptual, bloque B: cartelera y cines

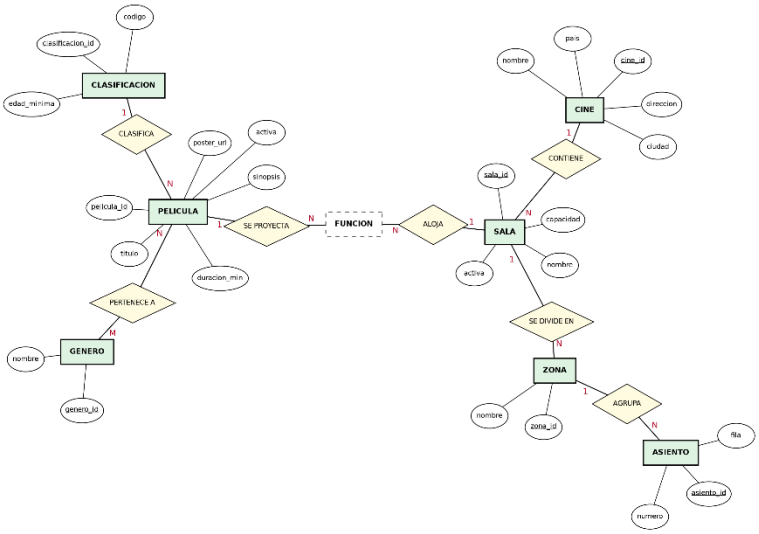


Figura 4
DER conceptual, bloque C: funciones, tarifas y estado de asientos

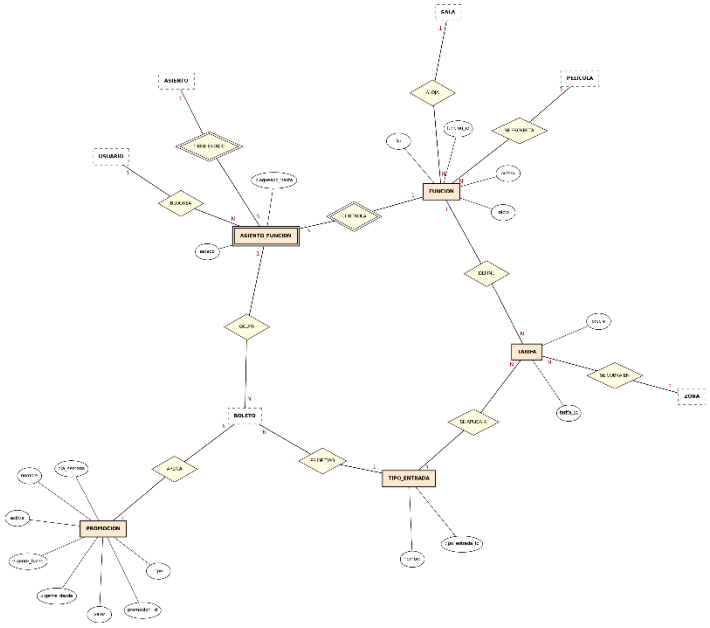


Figura 5
DER conceptual, bloque D: venta, pagos, boletos y reembolsos

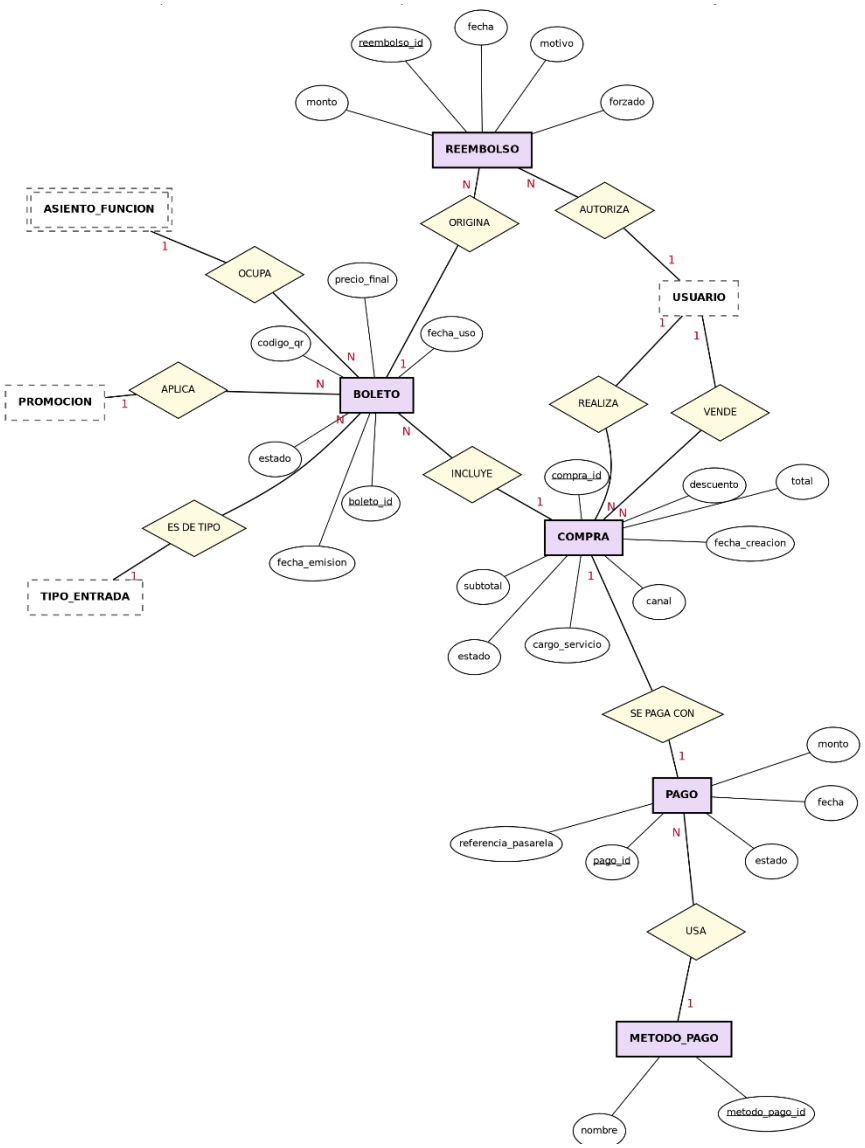
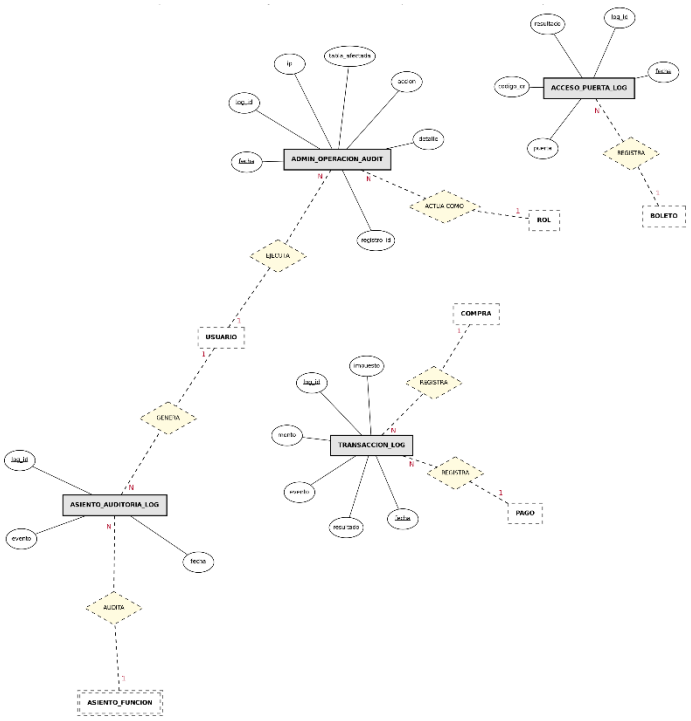


Figura 6
DER conceptual, bloque E: logs de auditoría



4.2 Diseño lógico

Diagrama del modelo

Figura 7
Diagrama del modelo lógico

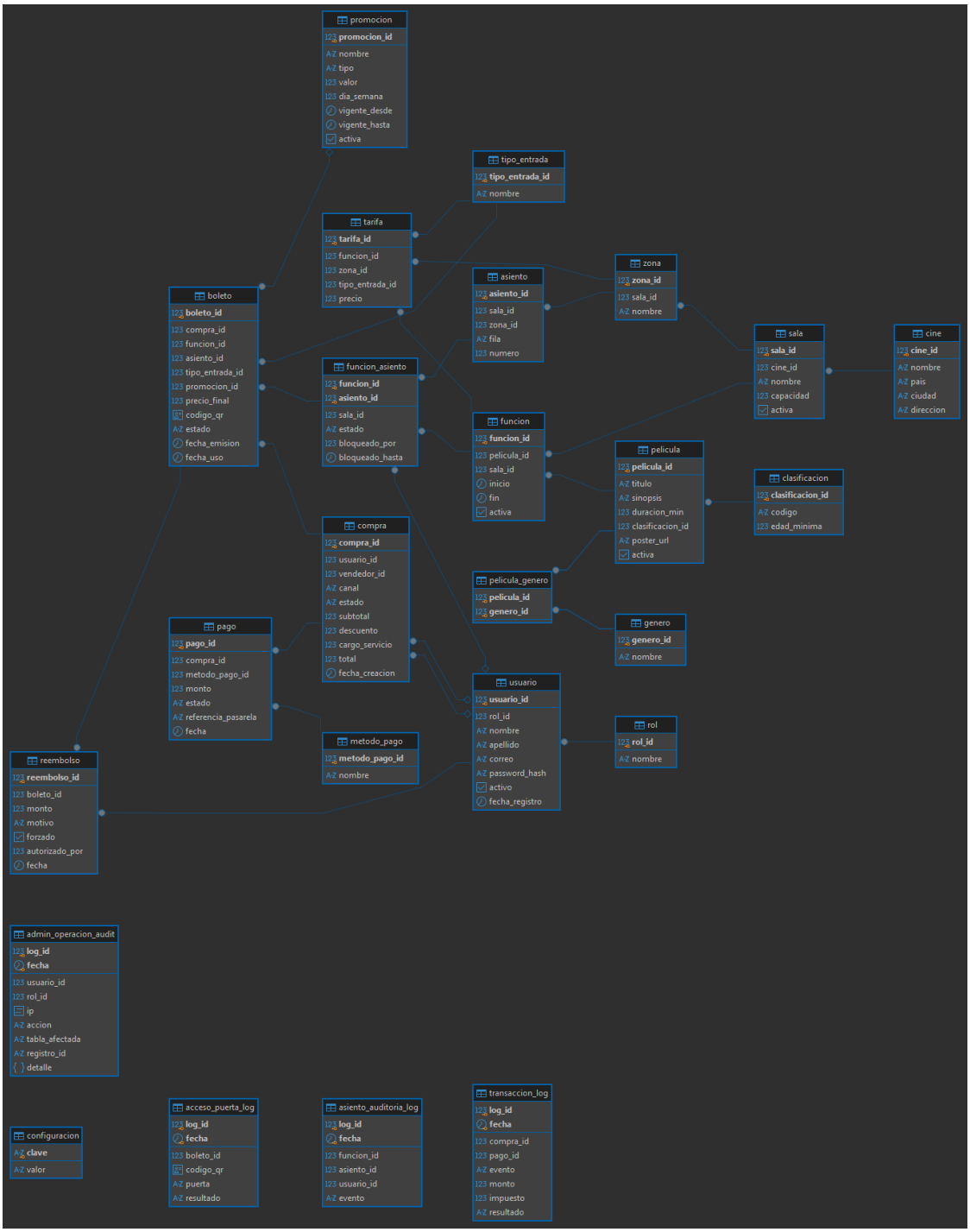


Tabla de relaciones y cardinalidades

Tabla origen	Cardinalidad	Tabla destino	Descripción
rol	1 : N	usuario	Un rol se asigna a muchos usuarios
clasificacion	1 : N	pelicula	Una clasificación aplica a muchas películas
pelicula	1 : N	pelicula_genero	Una película tiene varios géneros
genero	1 : N	pelicula_genero	Un género describe varias películas
cine	1 : N	sala	Un cine contiene varias salas
sala	1 : N	zona	Una sala se divide en zonas

Tabla origen	Cardinalidad	Tabla destino	Descripción
sala	1 : N	asiento	Una sala tiene muchos asientos
zona	1 : N	asiento	Una zona agrupa asientos
pelicula	1 : N	funcion	Una película se proyecta en varias funciones
sala	1 : N	funcion	Una sala aloja varias funciones
funcion	1 : N	tarifa	Una función define varias tarifas
zona	1 : N	tarifa	Una zona tiene precios distintos por tipo de entrada
tipo_entrada	1 : N	tarifa	Un tipo de entrada tiene tarifas por función y zona
funcion	1 : N	funcion_asiento	Una función controla el estado de cada asiento
asiento	1 : N	funcion_asiento	Un asiento tiene un estado por cada función
usuario	1 : N	funcion_asiento	Un usuario puede bloquear asientos temporalmente
usuario	1 : N	compra	Un usuario realiza compras (comprador o taquillero vendedor)
compra	1 : 1	pago	Una compra se paga con un único pago
metodo_pago	1 : N	pago	Un método de pago se usa en muchos pagos
compra	1 : N	boleto	Una compra incluye uno o más boletos
funcion_asiento	1 : N	boleto	Un asiento de una función se ocupa con un boleto (a lo sumo uno no anulado)
tipo_entrada	1 : N	boleto	Un boleto es de un tipo de entrada
promocion	1 : N	boleto	Un boleto puede tener una promoción aplicada
boleto	1 : N	reembolso	Un boleto puede tener reembolsos
usuario	1 : N	reembolso	Un usuario autoriza reembolsos

Tablas del modelo, bloques A a E

Bloque A: usuarios y configuración

Tabla	Columnas	Restricciones
rol	rol_id (PK), nombre	nombre único
usuario	usuario_id (PK), rol_id (FK), nombre, apellido, correo, password_hash, activo, fecha_registro	correo único
configuracion	clave (PK), valor	Parámetros del sistema (RN-04, RN-05, RN-16, RN-19)

Bloque B: cartelera y cines

Tabla	Columnas	Restricciones
clasificacion	clasificacion_id (PK), codigo, edad_minima	codigo único; edad_minima >= 0
genero	genero_id (PK), nombre	nombre único
pelicula	pelicula_id (PK), titulo, sinopsis, duracion_min, clasificacion_id (FK), poster_url, activa	título, sinopsis y clasificación obligatorios (RN-02); duracion_min > 0
pelicula_genero	pelicula_id (FK), genero_id (FK)	PK compuesta (relación N:M entre película y género)
cine	cine_id (PK), nombre, pais, ciudad, direccion	
sala	sala_id (PK), cine_id (FK), nombre, capacidad, activa	único (cine_id, nombre); capacidad > 0
zona	zona_id (PK), sala_id (FK), nombre	único (sala_id, nombre); único (zona_id, sala_id)
asiento	asiento_id (PK), sala_id, zona_id, fila, numero	FK compuesta (zona_id, sala_id) hacia zona; único (sala_id, fila, numero); único (asiento_id, sala_id)

Bloque C: funciones, tarifas y estado de asientos

Tabla	Columnas	Restricciones
funcion	funcion_id (PK), pelicula_id (FK), sala_id (FK), inicio, fin, activa	fin > inicio; único (funcion_id, sala_id); restricción EXCLUDE: sin traslape de funciones activas en la misma sala

Tabla	Columnas	Restricciones
tipo_entrada	tipo_entrada_id (PK), nombre	nombre único
tarifa	tarifa_id (PK), funcion_id (FK), zona_id (FK), tipo_entrada_id (FK), precio	único (funcion_id, zona_id, tipo_entrada_id); precio >= 0
promocion	promocion_id (PK), nombre, tipo, valor, dia_semana, vigente_desde, vigente_hasta, activa	tipo en ('2x1', 'porcentaje'); valor entre 0 y 100; vigente_hasta >= vigente_desde
funcion_asiento	funcion_id y asiento_id (PK compuesta), sala_id, estado, bloqueado_por (FK), bloqueado_hasta	estado en (disponible, bloqueado, vendido); FK compuestas a funcion y asiento por sala_id; un asiento bloqueado exige dueño y hora de expiración

Bloque D: venta, pagos, boletos y reembolsos

Tabla	Columnas	Restricciones
metodo_pago	metodo_pago_id (PK), nombre	nombre único
compra	compra_id (PK), usuario_id (FK, nulo), vendedor_id (FK, nulo), canal, estado, subtotal, descuento, cargo_servicio, total, fecha_creacion	total = subtotal - descuento + cargo_servicio (RN-08); canal online exige usuario; canal taquilla exige vendedor
pago	pago_id (PK), compra_id (FK), metodo_pago_id (FK), monto, estado, referencia_pasarela, fecha	compra_id único (relación 1:1); sin datos de tarjeta ni CVV (RN-09)
boleto	boleto_id (PK), compra_id (FK), funcion_id, asiento_id, tipo_entrada_id (FK), promocion_id (FK, nulo), precio_final, codigo_qr (nulo), estado, fecha_emision, fecha_uso	FK compuesta (funcion_id, asiento_id) hacia funcion_asiento; codigo_qr único; estado válido o usado exige QR (RN-11, RN-12); índice único parcial (funcion_id, asiento_id) donde estado <> anulado
reembolso	reembolso_id (PK), boleto_id (FK), monto, motivo, forzado, autorizado_por (FK), fecha	monto > 0

Bloque E: logs de auditoría (retención de 5 años)
Cuatro tablas particionadas por año (2026 a 2031), con llave primaria (log_id, fecha) y sin llaves foráneas, para que escribir en ellas sea rápido.

Tabla	Columnas principales
transaccion_log	log_id, compra_id, pago_id, evento, monto, impuesto, resultado, fecha
asiento_auditoria_log	log_id, funcion_id, asiento_id, usuario_id, evento, fecha
acceso_puerta_log	log_id, boleto_id, codigo_qr, puerta, resultado, fecha
admin_operacion_audit	log_id, usuario_id, rol_id, ip, accion, tabla_afectada, registro_id, detalle (JSONB), fecha

Normalización

Todas las tablas están en tercera forma normal (3FN): cada columna que no es llave depende solo de la llave de su tabla. Hay dos redundancias deliberadas, protegidas con llaves compuestas:

- asiento.sala_id (se podría deducir de zona): permite la restricción única (sala_id, fila, numero), y la llave foránea compuesta evita que la zona pertenezca a otra sala.
- funcion_asiento.sala_id (se podría deducir de funcion y asiento): garantiza que un asiento solo se asigne a funciones de su misma sala.

Supuestos de diseño

El enunciado no define estos puntos, por lo que se tomaron las siguientes decisiones:

1. Las tarifas se definen por función, zona y tipo de entrada.
2. Una compra tiene un solo pago (relación 1:1); los intentos fallidos quedan registrados en transaccion_log.
3. La puerta de acceso se registra como texto en acceso_puerta_log.
4. Los logs no usan llaves foráneas, por rendimiento de escritura.
5. Los valores de configuracion (máximo de asientos por compra, minutos de bloqueo, tolerancia de ingreso, límite de reembolso) son los ejemplos del enunciado y pueden cambiarse.

6. El estado "reservado" de RN-03 se trata igual que "bloqueado".
7. El DER conceptual muestra solo la cardinalidad máxima; la participación opcional se refleja en las columnas que admiten NULL.
8. El trigger de RN-18 bloquea UPDATE y DELETE sobre sala y zona, e INSERT, UPDATE y DELETE sobre asiento, cuando la sala tiene boletos reservados o válidos en funciones que aún no terminan. No bloquea el INSERT de una zona nueva: agregar una zona sin asientos no cambia ningún boleto vendido, pero es una brecha frente a la letra de la regla, que se cerraría agregando INSERT al trigger trg_zona_proteger.

4.3 Diseño físico

Motor. La base corre en PostgreSQL 18. Lo elegimos porque cubre lo que el proyecto necesita: transacciones ACID, índices únicos parciales para evitar la doble venta, restricciones EXCLUDE para que una sala no tenga funciones traslapadas, particionamiento por fecha para las bitácoras y JSONB para el detalle de las operaciones administrativas.

Índices.

Índice	Tabla y columnas	Tipo	Para qué sirve
uq_boleto_asiento_activo	boleto (funcion_id, asiento_id), solo donde estado <> anulado	Único parcial	Impide vender dos veces el mismo asiento y acelera la ocupación por función
boleto_codigo_qr_key	boleto (codigo_qr)	Único	QR único (RN-12) y validación rápida en puerta
funcion_asiento_pkey	funcion_asiento (funcion_id, asiento_id)	Llave primaria	Mapa de asientos y ocupación de una función
idx_funcion_inicio_activa	funcion (inicio), solo donde activa	Parcial	Cartelera filtrada por fecha
idx_funcion_asiento_bloqueados	funcion_asiento (bloqueado_hasta), solo donde estado = bloqueado	Parcial	Encontrar los bloqueos vencidos (fn_liberar_bloqueos_expirados)
zona_zona_id_sala_id_key	zona (zona_id, sala_id)	Único	Respalda la llave foránea compuesta de asiento
idx_boleto_compra	boleto (compra_id)	B-tree	Buscar los boletos de una compra (pago y anulación de compras)
idx_compra_usuario	compra (usuario_id)	B-tree	Buscar las compras de un cliente (Mis boletos)
idx_boleto_emision_vendidos	boleto (fecha_emision) con funcion_id incluido, solo donde estado es valido o usado	Parcial	Índice cubriente para el top 5 de películas de la semana (consulta analítica 2), pensado para no leer la tabla

Particionamiento y retención. Las cuatro bitácoras están particionadas por año sobre la columna fecha (de 2026 a 2031). Su llave primaria es (log_id, fecha) porque en PostgreSQL la columna de partición tiene que formar parte de la llave. Cuando una partición cumple los 5 años de retención, se desvincula de la tabla, se exporta a almacenamiento de archivo y se elimina de la base viva, sin tocar las demás particiones.

4.4 Seguridad y roles

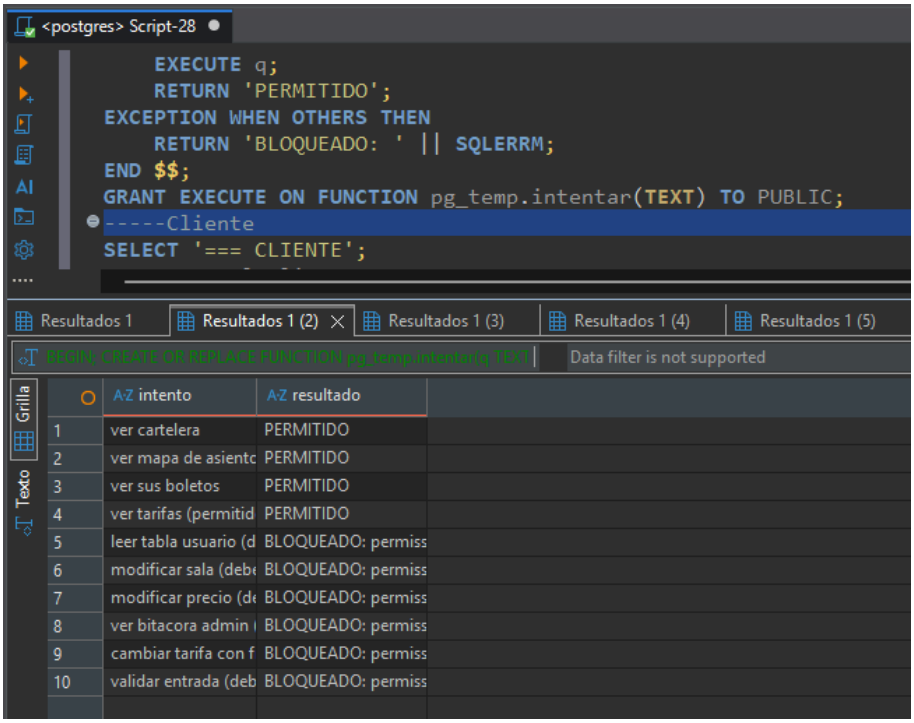
El control de acceso se hace dentro de la base de datos, con roles de PostgreSQL y GRANT, y no solo en la aplicación. Cada rol recibe lo mínimo que necesita. Las operaciones sensibles pasan por funciones (fn_pagar_compra, fn_cambiar_tarifa, fn_otorgar_cortesia y fn_validar_entrada) y a los roles se les niega el acceso directo a las tablas. Para comprobarlo, el script 11_rbac_demo.sql cambia a cada rol e intenta operaciones permitidas y prohibidas.

Matriz de privilegio mínimo

Módulo	Cliente	Taquillero	Operador de sala	Contador	Administrador
Catálogo y cartelera	Lectura	Lectura	Lectura	Lectura	Lectura y escritura
Selección de asientos y pago	Compra online	Venta presencial	Sin acceso	Sin acceso	Configura reglas y precios

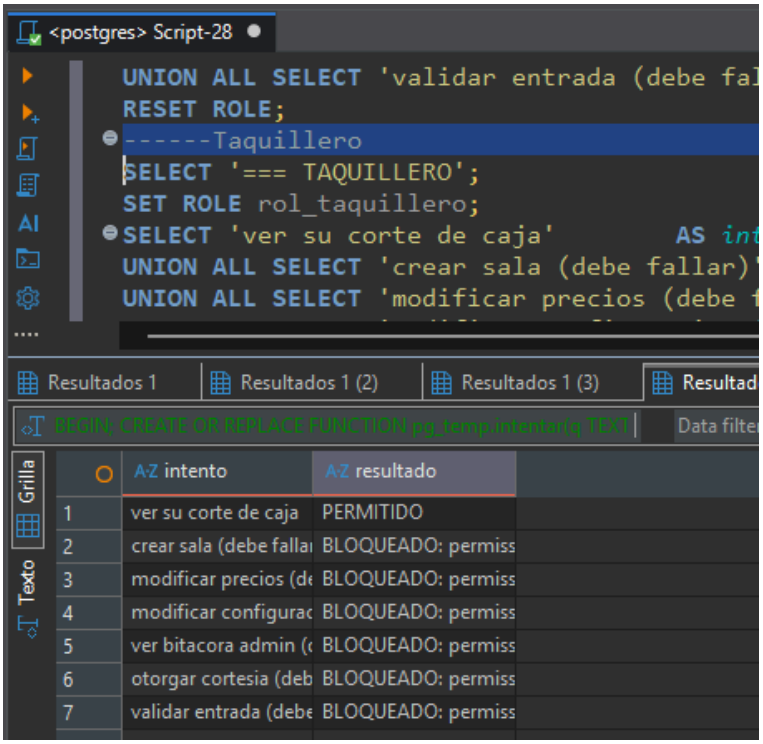
Módulo	Cliente	Taquillero	Operador de sala	Contador	Administrador
Emisión y acceso (QR)	Ve sus QR	Emite boletos	Valida QR en puerta	Sin acceso	Incidencias y reasignaciones
Reportes y finanzas	Sin acceso	Su corte de caja	Sin acceso	Lectura y exportación	Lectura total
Salas y funciones	Sin acceso	Sin acceso	Sin acceso	Sin acceso	Lectura y escritura (limitado por RN-18)
Auditoría y roles	Sin acceso	Sin acceso	Sin acceso	Sin acceso	Lectura total

Figura 8
Prueba de privilegios del rol cliente



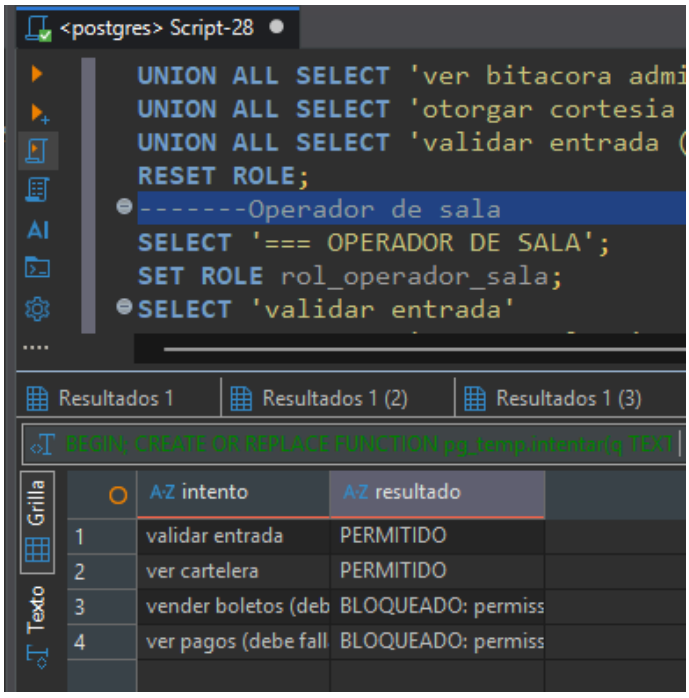
Cliente: puede ver la cartelera, el mapa de asientos, sus boletos y las tarifas. Se le niega leer la tabla usuario, modificar salas o precios, ver la bitácora, cambiar tarifas y validar entradas.

Figura 9
Prueba de privilegios del rol taquillero



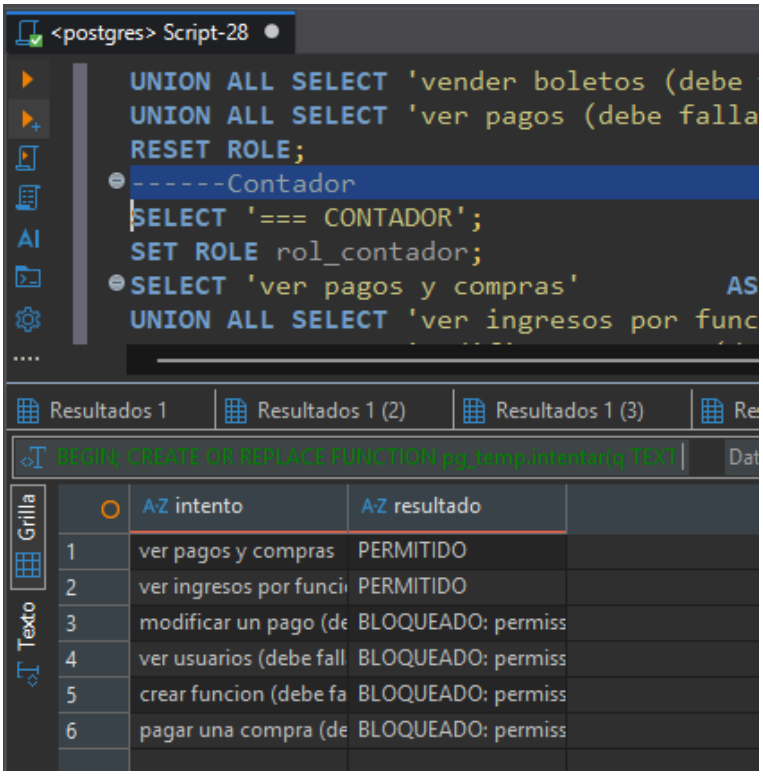
Taquillero: puede ver su corte de caja. Se le niega crear salas, modificar precios o configuración, ver la bitácora, otorgar cortesías y validar entradas.

Figura 10
Prueba de privilegios del rol operador de sala



Operador de sala: puede validar entradas y ver la cartelera. Se le niega vender boletos y ver pagos.

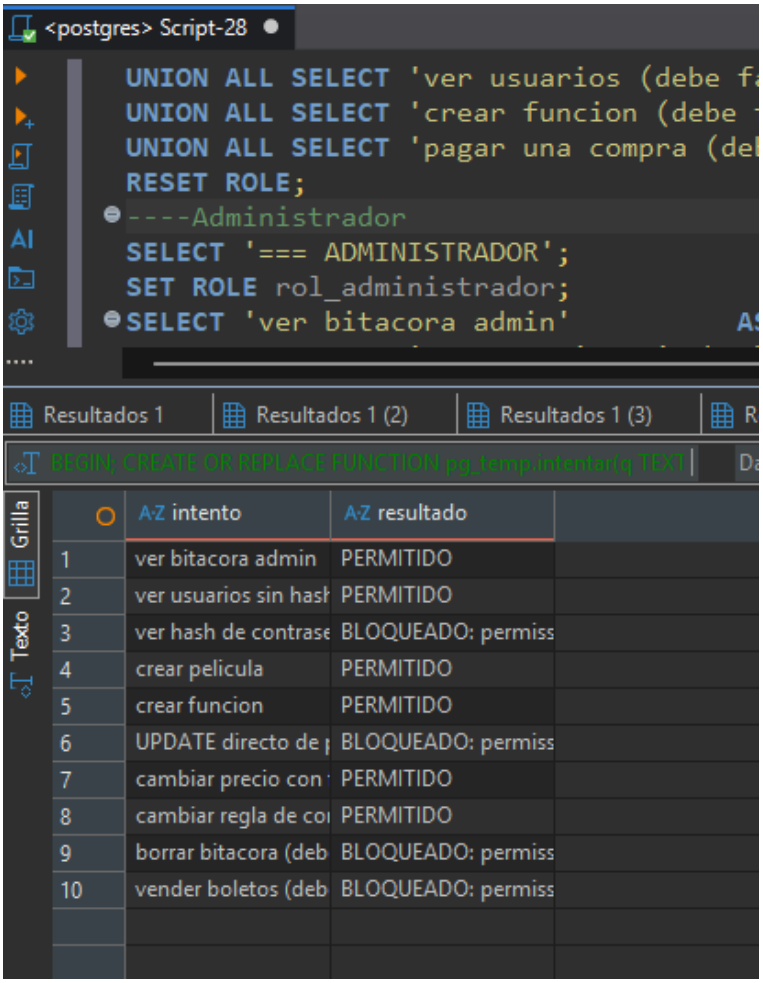
Figura 11
Prueba de privilegios del rol contador



Contador: puede ver pagos, compras e ingresos por función. Se le niega modificar pagos, ver usuarios, crear funciones y pagar compras

.

Figura 12
Prueba de privilegios del rol administrador



Administrador: puede ver la bitácora y los usuarios sin el hash, y crear películas y funciones. No puede ver el hash de la contraseña, actualizar precios directamente, borrar la bitácora ni vender boletos. Sí puede cambiar una tarifa y una regla de configuración, pero solo con las funciones fn_cambiar_tarifa y fn_cambiar_configuracion, que exigen un motivo y dejan una fila en la bitácora.

Cómo se cumple cada regla de negocio

Regla	Cómo se cumple
RN-01	Vista v_cartelera: solo muestra funciones activas, de películas y salas activas, con inicio futuro (inicio > now())
RN-02	Columnas obligatorias (NOT NULL) en pelicula: título, sinopsis y clasificación
RN-03	Estado de cada asiento por función en funcion_asiento y vista v_mapa_asientos
RN-04	Valor configurable max_asientos_por_compra en la tabla configuracion; lo validan fn_bloquear_asiento (asientos bloqueados por usuario) y fn_crear_compra (asientos por compra)
RN-05	bloqueado_hasta, un CHECK que exige dueño y hora de expiración, y fn_liberar_bloqueos_expirados
RN-06	tipo_entrada_id en boleto; el documento se revisa en persona en la puerta
RN-07	Tabla promocion (tipo, valor y vigencia) y boleto.promocion_id
RN-08	CHECK en compra: total = subtotal - descuento + cargo_servicio
RN-09	pago sin columnas de tarjeta ni CVV; solo referencia_pasarela
RN-10	fn_pagar_compra: si la pasarela rechaza el pago, lo registra como rechazado, anula los boletos y libera los asientos en la misma transacción (fn_anular_compra_pendiente)
RN-11	CHECK en boleto: un boleto válido o usado exige codigo_qr
RN-12	codigo_qr único (boleto_codigo_qr_key) y llave foránea compuesta (funcion_id, asiento_id)
RN-13	Vista v_mis_boletos
RN-14	fn_validar_entrada: bloquea la fila del boleto, lo marca como usado en el primer escaneo y rechaza el segundo con “Entrada ya utilizada”; todo queda en acceso_puerta_log

Regla	Cómo se cumple
RN-15	fn_validar_entrada compara la función del boleto con la que se atiende (rechazado_funcion) y revisa el horario (rechazado_horario)
RN-16	Tolerancia guardada en configuracion (30 minutos antes y 15 después) y revisada por fn_validar_entrada
RN-17	Roles de PostgreSQL con GRANT; las operaciones sensibles solo se hacen por funciones
RN-18	Triggers trg_sala_proteger, trg_zona_proteger y trg_asiento_proteger (fn_proteger_sala_con_ventas): impiden eliminar o cambiar la distribución de una sala con boletos reservados o válidos en funciones activas que aún no terminan
RN-19	Límite horas_limite_reembolso en configuracion; fn_reembolsar_boleto valida el plazo, anula el boleto, libera el asiento y registra el reembolso en la tabla reembolso
RN-20	Cuatro bitácoras particionadas por año; el trigger trg_log_inmutable impide modificar o borrar sus filas; admin_operacion_audit guarda fecha, IP, rol y usuario

5. Implementación y pruebas

5.1 DDL y restricciones

El esquema se crea con el script 01_ddl.sql: 25 tablas (21 de operación y las 4 bitácoras de auditoría) y los datos base que el sistema necesita para funcionar (roles, tipos de entrada, métodos de pago y parámetros de configuración). Las reglas del negocio se protegen en la propia base de datos con restricciones, de modo que ninguna aplicación pueda saltárselas:

- **Llaves y unicidad.** Llave primaria en todas las tablas y UNIQUE en el correo, el código QR, (sala_id, fila, numero) y (funcion_id, zona_id, tipo_entrada_id).
- **Llaves foráneas compuestas.** asiento hacia zona por (zona_id, sala_id), funcion_asiento hacia funcion y asiento por sala_id, y boleto hacia funcion_asiento por (funcion_id, asiento_id). Así un asiento solo puede usarse en funciones de su propia sala y un boleto solo apunta a un asiento que existe en esa función.
- **Restricciones CHECK.** Duración y capacidad mayores que cero, precio no negativo, porcentaje entre 0 y 100, estados permitidos, fin > inicio en las funciones, total = subtotal - descuento + cargo_servicio (RN-08), asiento bloqueado con dueño y hora de expiración (RN-05) y boleto válido o usado con código QR (RN-11).
- **Sin traslape de funciones.** Una restricción EXCLUDE (con la extensión btree_gist) impide dos funciones activas que se traslapen en la misma sala.
- **Anti doble venta.** El índice único parcial uq_boleto_asiento_activo permite un solo boleto no anulado por función y asiento.
- **Pagos sin datos de tarjeta.** La tabla pago no tiene columnas para el número de tarjeta ni el CVV (RN-09).
- **Bitácoras.** Cuatro tablas particionadas por año (24 particiones, de 2026 a 2031) y sin llaves foráneas, para que escribir en ellas sea rápido.

5.2 Transacciones, triggers y procedimientos

La lógica crítica está en funciones y triggers de PostgreSQL (PL/pgSQL). Cada función corre dentro de una sola transacción: si algo falla, se deshace todo (ACID). Por ejemplo, fn_pagar_compra cambia el pago, los asientos, los boletos y las bitácoras juntos, o no cambia nada.

Objeto	Script	Qué hace
fn_bloquear_asiento	02	Bloquea un asiento por 2 minutos con un solo UPDATE atómico que solo actúa si está disponible. Valida el límite por usuario (RN-04) y registra el evento en asiento_auditoria_log.
fn_liberar_bloqueos_expirados	02	Libera los bloqueos vencidos (RN-05) y deja cada liberación en la bitácora.
trg_funcion_crear_asientos	02	Trigger: al crear una función, crea el estado “disponible” de cada asiento de la sala.
fn_crear_compra	03	Crea la compra y sus boletos reservados (sin QR) con el precio de la tarifa. Verifica que los asientos estén bloqueados por quien compra, aplica la promoción (porcentaje o 2x1) y calcula subtotal, descuento y total (RN-07, RN-08).
fn_pagar_compra	03	Pago simulado. Si se aprueba: asientos vendidos, QR emitido y compra pagada. Si se rechaza o el bloqueo venció: compra anulada y asientos liberados (RN-10, RN-11). Todo en una sola transacción.
fn_anular_compra_pendiente	03	Auxiliar: anula los boletos reservados y libera los asientos de una compra pendiente.
fn_validar_entrada	04	Valida el QR en la puerta: un solo uso, función correcta y tolerancia de horario (RN-14, RN-15, RN-16). Siempre deja un registro en acceso_puerta_log.
fn_reembolsar_boleto	04	Reembolso dentro del plazo (RN-19) o forzado por un administrador: anula el boleto, libera el asiento y registra el reembolso.

Objeto	Script	Qué hace
trg_sala_proteger, trg_zona_proteger, trg_asiento_proteger	05	Triggers que impiden modificar o eliminar una sala con ventas activas (RN-18).
fn_cambiar_asiento, fn_otorgar_cortesia, fn_cambiar_tarifa, fn_cambiar_configuracion	05 y 07	Operaciones manuales del administrador (RN-17, RN-20): exigen rol de administrador y motivo, y dejan una fila en admin_operacion_audit con fecha, IP, rol y usuario.
trg_log_inmutable	05	Trigger que impide UPDATE y DELETE sobre las cuatro bitácoras (RN-20).
fn_login_usuario	12	Inicio de sesión con la contraseña cifrada con bcrypt (pgcrypto). Nunca devuelve el hash.

Todas las funciones usan SECURITY DEFINER con un search_path fijo y se les quitó el permiso a PUBLIC; cada rol solo ejecuta las que le corresponden (script 07).

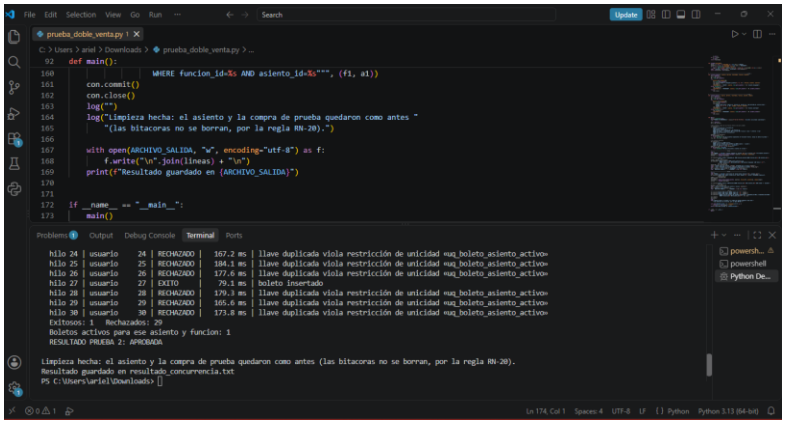
5.3 Prueba de concurrencia

Para comprobar que no se puede vender dos veces el mismo asiento, el script prueba_doble_venta.py abre 30 conexiones a la base, cada una con un cliente distinto, y las suelta todas al mismo tiempo con una barrera de sincronización (threading.Barrier). Hace dos pruebas:

- **Prueba 1, bloqueo del asiento (fn_bloquear_asiento).** Los 30 clientes intentan bloquear el mismo asiento. Un hilo tuvo éxito y los otros 29 fueron rechazados con el mensaje “El asiento no esta disponible”, porque la función hace un solo UPDATE que solo actúa si el asiento sigue disponible. El asiento quedó bloqueado por un único usuario y asiento_auditoria_log registró una sola fila “bloqueado”.
- **Prueba 2, inserción del boleto.** Los 30 hilos intentan insertar un boleto para la misma función y el mismo asiento. Un hilo tuvo éxito y los otros 29 fueron rechazados por la restricción uq_boleto_asiento_activo (llave duplicada). La consulta de verificación confirma que ese asiento tiene un único boleto activo.

Las dos pruebas dieron como resultado APROBADA. Al terminar, el script borra la compra de prueba y deja el asiento como estaba (las bitácoras no se borran, por la regla RN-20). La salida completa está en resultado_concurrencia.txt, dentro del repositorio, y la captura de la ejecución se muestra a continuación.

Figura 13
Salida de la prueba de concurrencia (prueba_doble_venta.py)



5.4 Consultas analíticas

Las cuatro consultas analíticas de la gerencia están en el script 08_consultas_analiticas.sql y son solo SELECT, por lo que no modifican datos. Se ejecutaron sobre la base con los datos de prueba de la sección 5.5 (unos 202 mil boletos). Cada figura muestra la consulta y su resultado.

Figura 14

Consulta 1: ocupación y recaudación por función

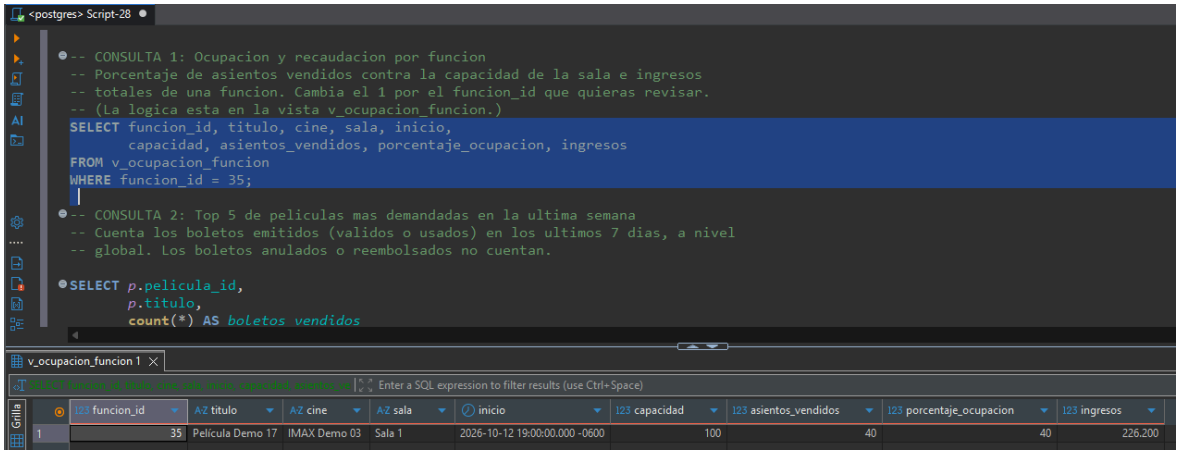


Figura 15

Consulta 2: top 5 de películas de la última semana

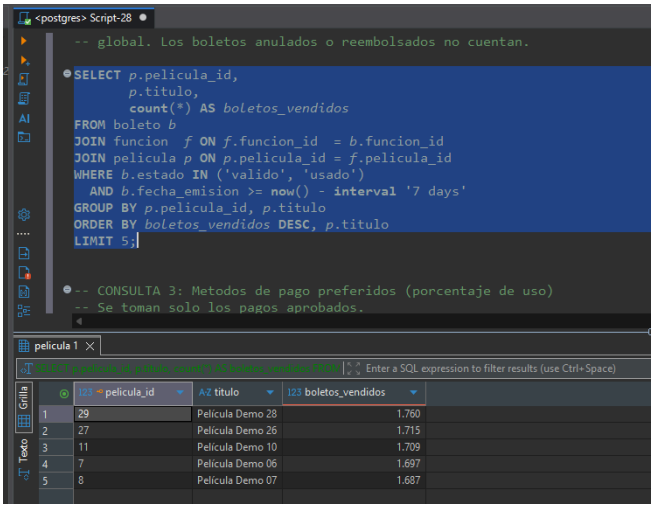


Figura 16

Consulta 3: métodos de pago preferidos

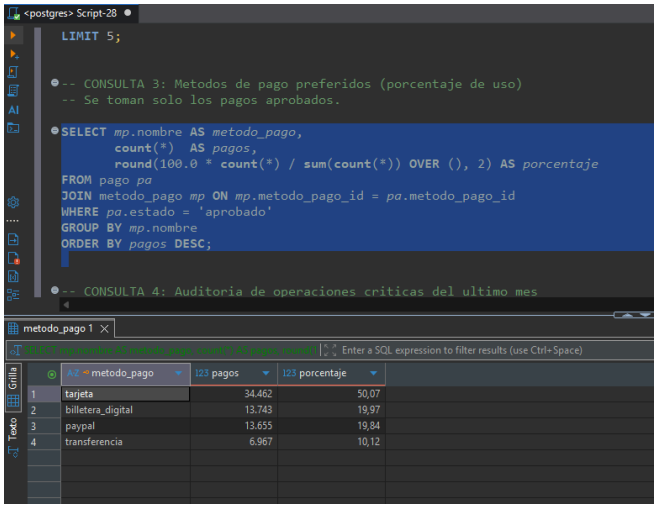


Figura 17
Consulta 4a: operaciones manuales por administrador en el último mes

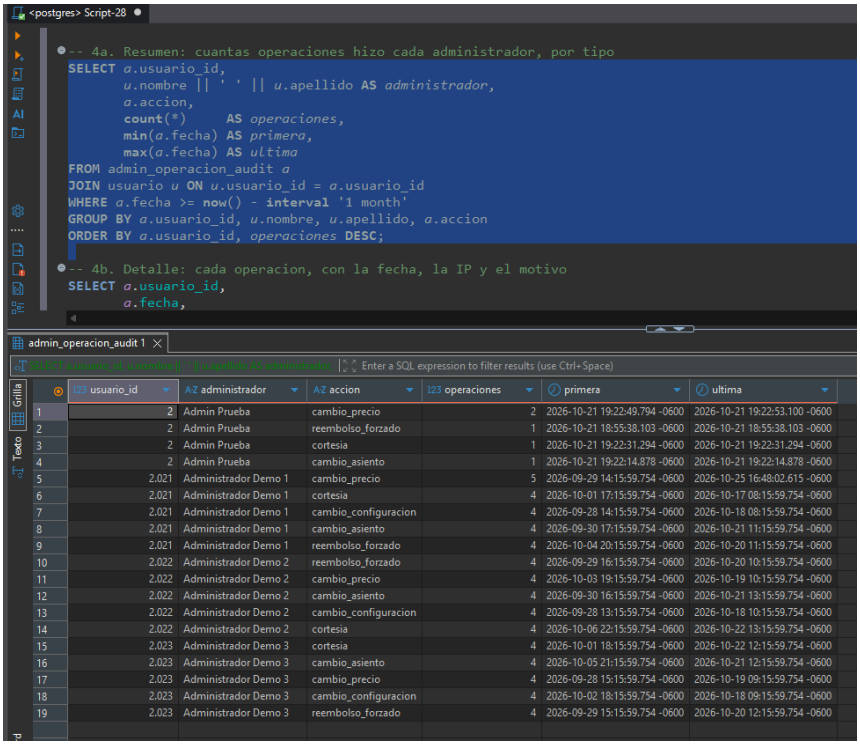
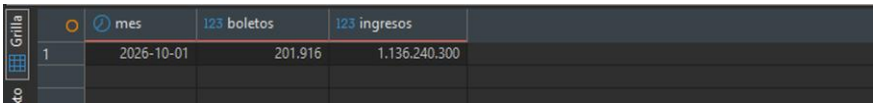


Figura 18
Corte de ingresos por mes



5.5 EXPLAIN ANALYZE

Para ver qué tanto ayudan los índices, llenamos la base con datos de prueba (unos 202 mil boletos, además de otras tablas) y corrimos EXPLAIN (ANALYZE, BUFFERS) antes y después de crearlos. Sin índices, buscar los boletos de una compra obligaba a leer la tabla completa y descartar 201,915 filas, y tardaba 32.4 ms. Revisar los bloqueos vencidos era peor: un Parallel Seq Scan sobre toda funcion_asiento, de 169.9 ms. Después creamos los índices, ejecutamos ANALYZE y VACUUM (ANALYZE) y repetimos las mediciones. Los boletos de una compra pasaron a un Index Scan sobre idx_boleto_compra y bajaron a 6.3 ms, y los bloqueos vencidos ahora usan idx_funcion_asiento_bloqueados y tardan 2.2 ms. En el top 5 de películas, el Index Only Scan sobre idx_boleto_emision_vendidos redujo el tiempo de 165.2 ms a 56.6 ms, y «Mis boletos» bajó de 166.1 ms a 38.2 ms.

Comparación antes y después de los índices

N.º	Consulta	Plan antes	Tiempo antes	Índice que debería ayudar	Plan después	Tiempo después
1.1	Boletos de una compra	Seq Scan on boleto (descarta 201,915 filas)	32.4 ms	idx_boleto_compra (compra_id)	Index Scan using idx_boleto_compra	6.3 ms
1.2	Mis boletos (v_mis_boletos)	Gather (plan paralelo con Hash Join)	166.1 ms*	idx_compra_usuario (usuario_id)	Nested Loop con Index Scan using idx_compra_usuario	38.2 ms*
1.3	Top 5 películas de la semana	Gather Merge con Partial HashAggregate (plan paralelo)	165.2 ms*	idx_boleto_emision_vendidos (índice parcial que cubre la consulta)	Index Only Scan using idx_boleto_emision_vendidos	56.6 ms*
1.4	Bloqueos de asiento vencidos	Parallel Seq Scan on funcion_asiento	169.9 ms	idx_funcion_asiento_bloqueados (índice parcial por bloqueado_hasta)	Index Scan using idx_funcion_asiento_bloqueados	2.2 ms
1.5	Cartelera de un día	Seq Scan on funcion (descarta 4,202 filas)	1.9 ms*	idx_funcion_inicio_activa (inicio, solo funciones activas)	Index Scan using idx_funcion_inicio_activa	2.7 ms*

Consultas críticas después de crear los índices

Consulta	Plan e índice que usa	Ejecución
Mapa de asientos disponibles (v_mapa_asientos)	Bitmap Index Scan en funcion_asiento_pkey; Index Scan en asiento_asiento_id_sala_id_key y zona_zona_id_sala_id_key	0.83 ms*
Validar un QR en la puerta	Index Scan en boleto_codigo_qr_key (único)	0.070 ms
Ocupación y recaudación de una función (v_ocupacion_funcion)	Nested Loop Left Join con Index Scan en funcion_funcion_id_sala_id_key	0.61 ms*
Auditoría del último mes	Append con Subplans Removed: 5; solo recorre la partición admin_operacion_audit_2026	0.672 ms

El mapa de asientos de una función responde en unos 0.8 ms (tiempo del nodo raíz del plan), muy por debajo de la meta de 200 ms que fijamos en RNF-08.

Evidencia de los planes de ejecución

Las figuras siguientes muestran cada plan: primero las cinco consultas antes de crear los índices, luego la actualización de estadísticas, después las mismas cinco consultas con los índices y, al final, las consultas críticas.

Figura 19
Consulta 1.1 antes del índice: boletos de una compra

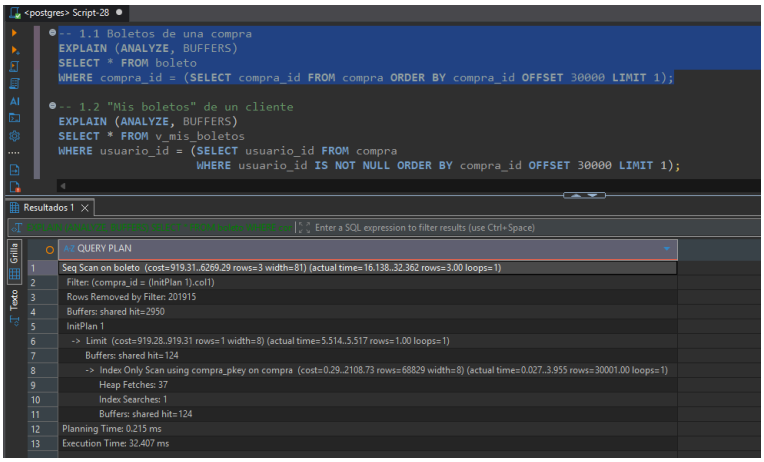


Figura 20
Consulta 1.2 antes del índice: Mis boletos

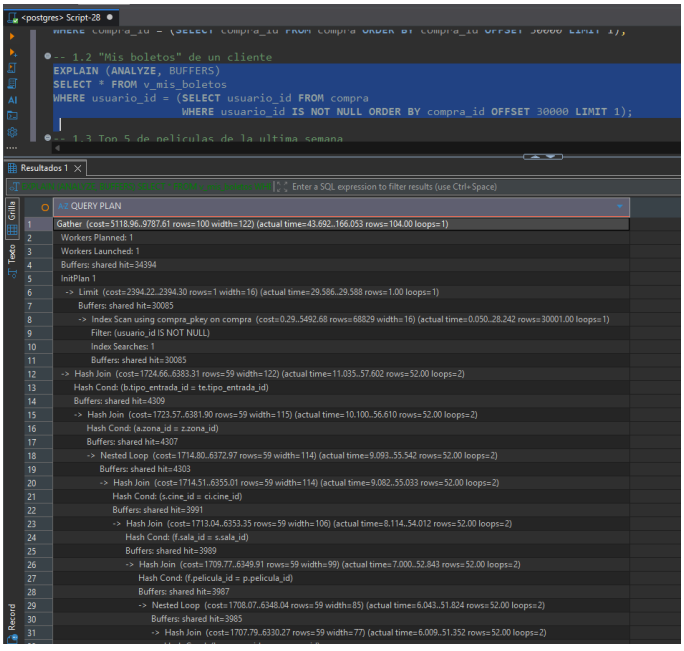


Figura 21
Consulta 1.3 antes del índice: top 5 de películas de la semana

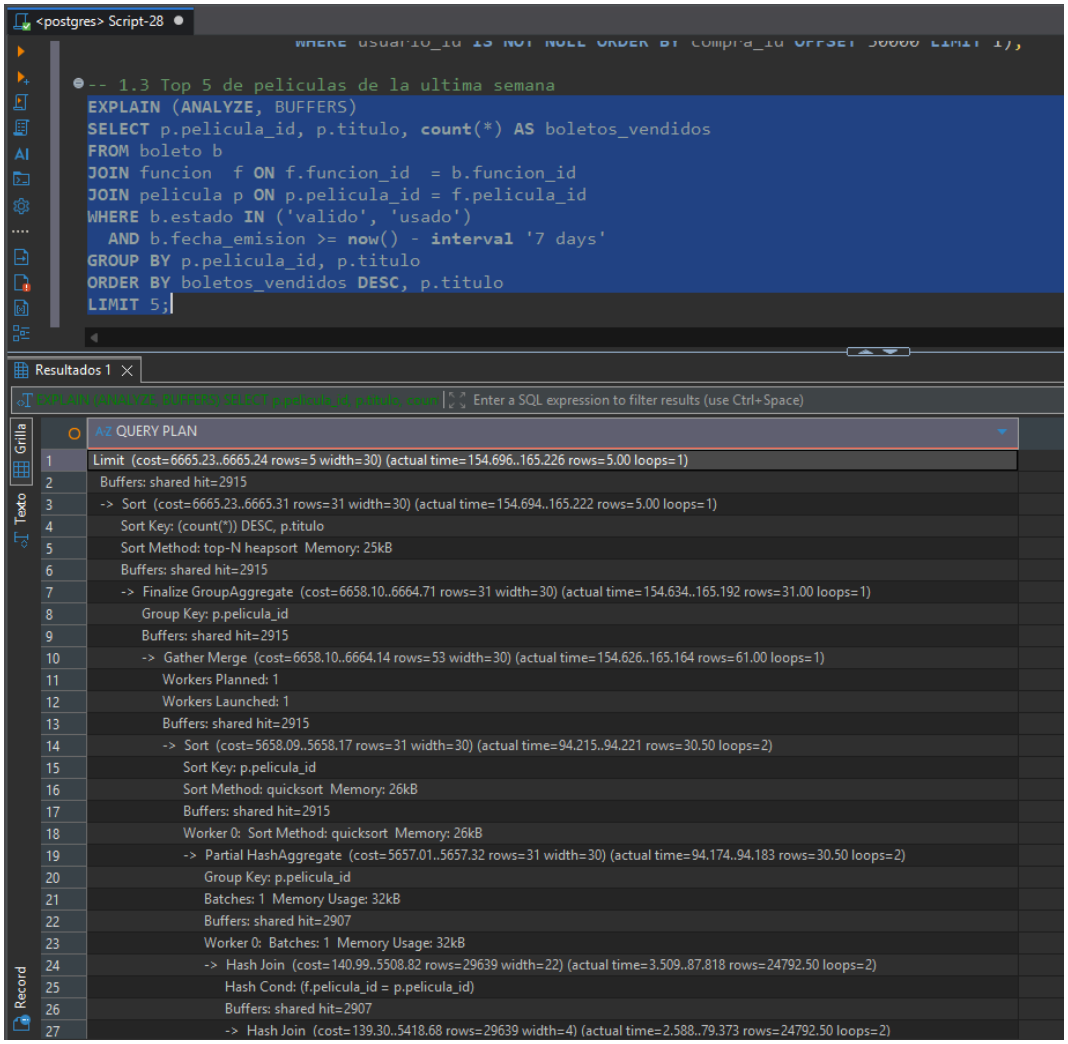


Figura 22
Consulta 1.4 antes del índice: bloqueos de asiento vencidos

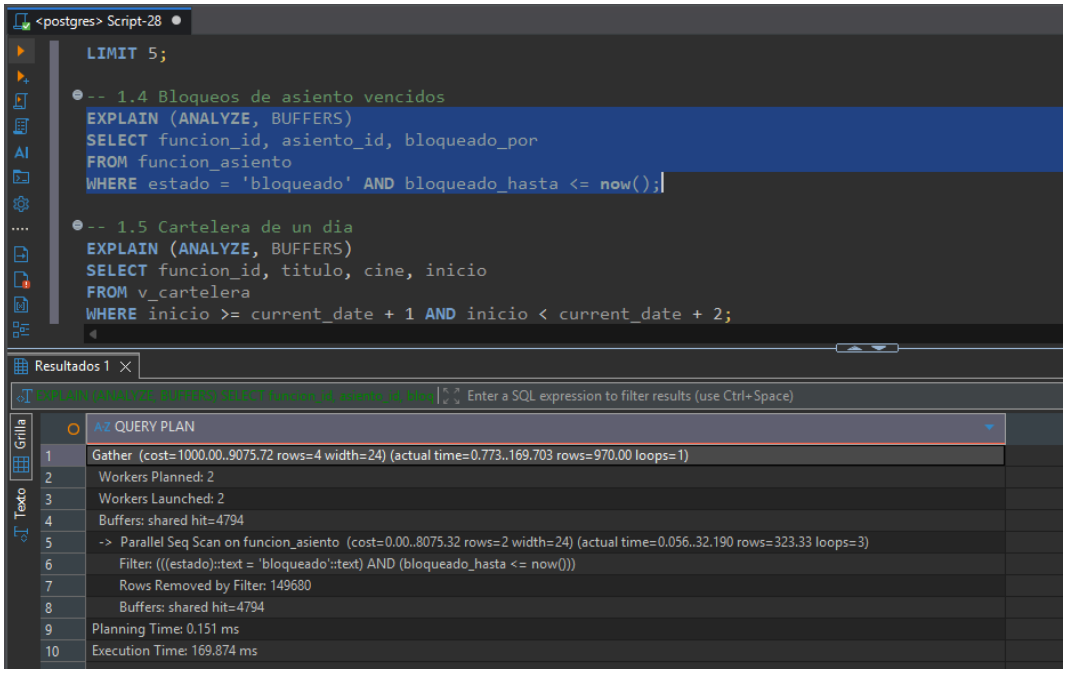


Figura 23
Consulta 1.5 antes del índice: cartelera de un día

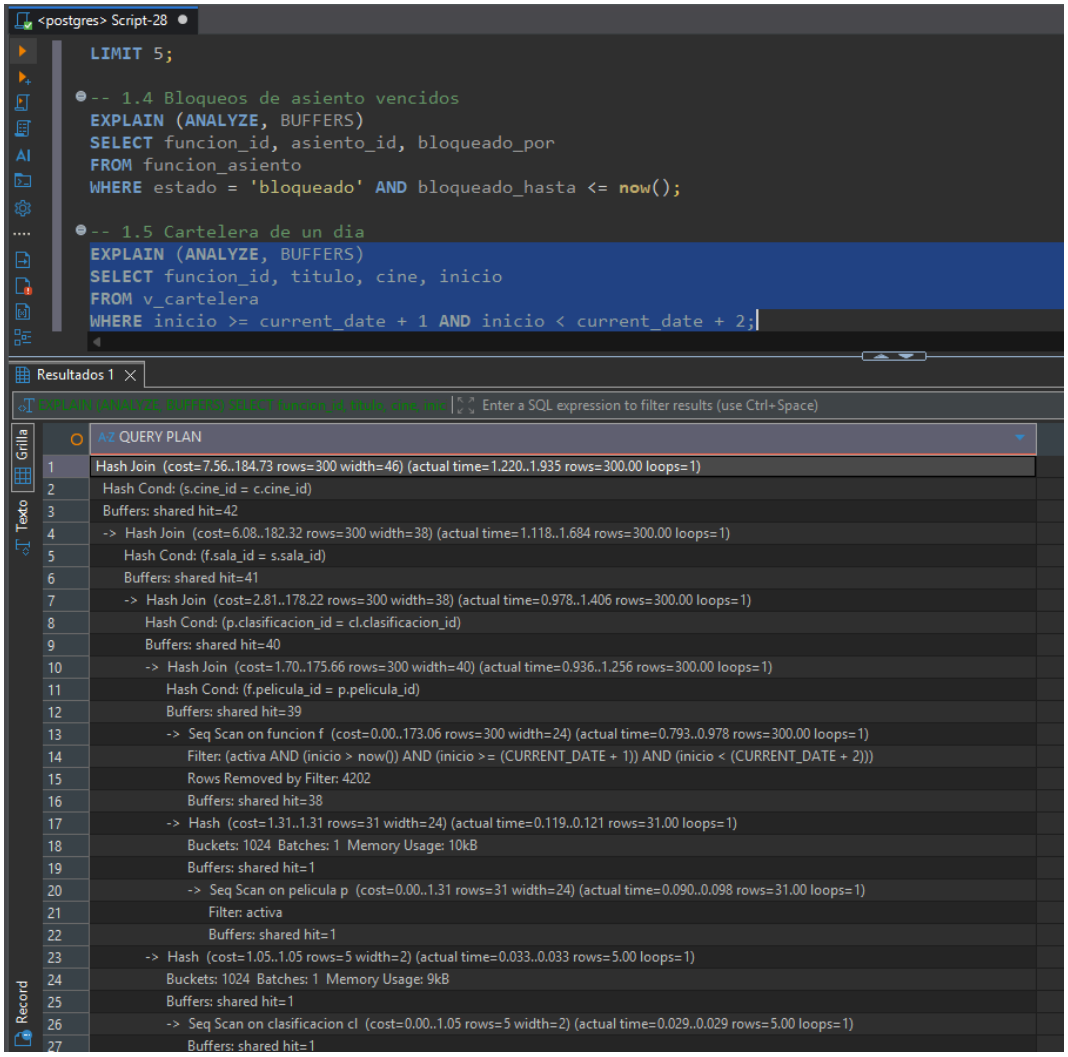


Figura 24
Actualización de estadísticas antes de repetir las mediciones

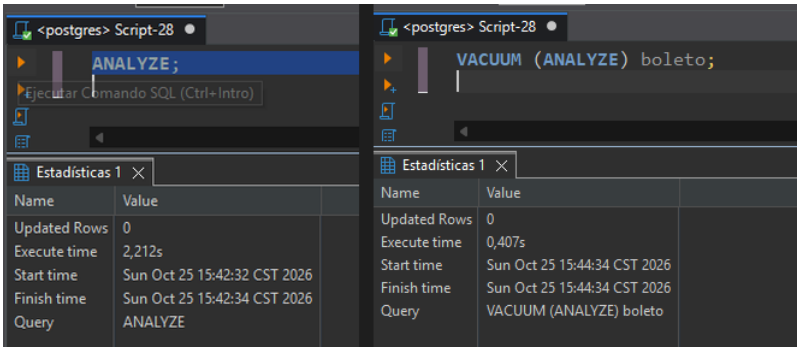


Figura 25
Consulta 1.1 después del índice: boletos de una compra

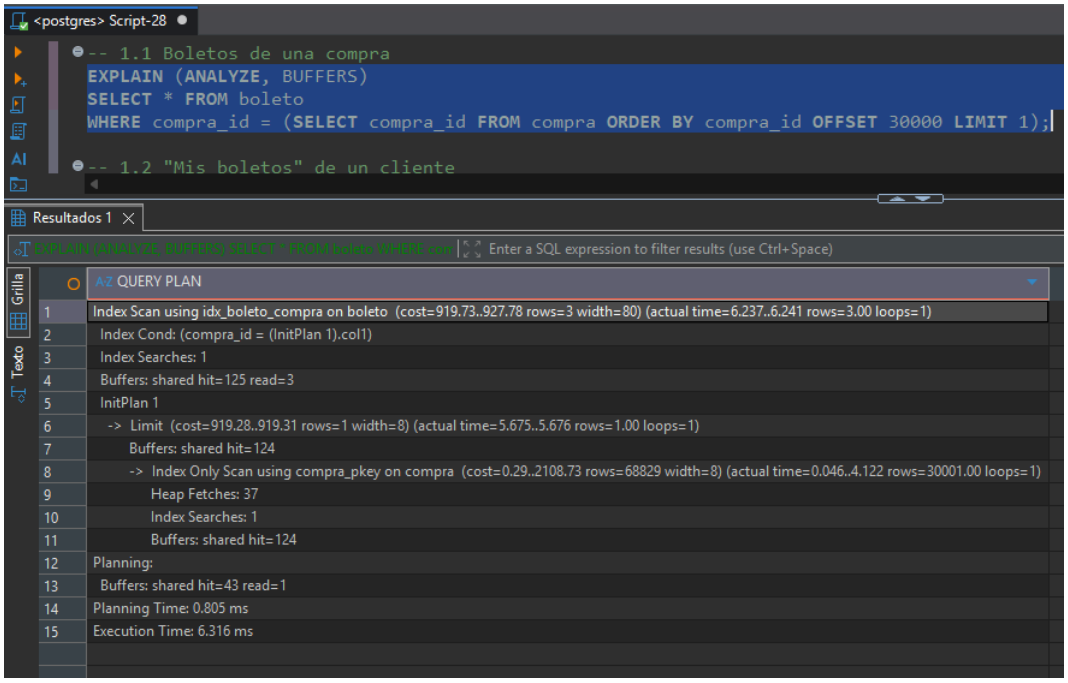


Figura 26
Consulta 1.2 después del índice: Mis boletos

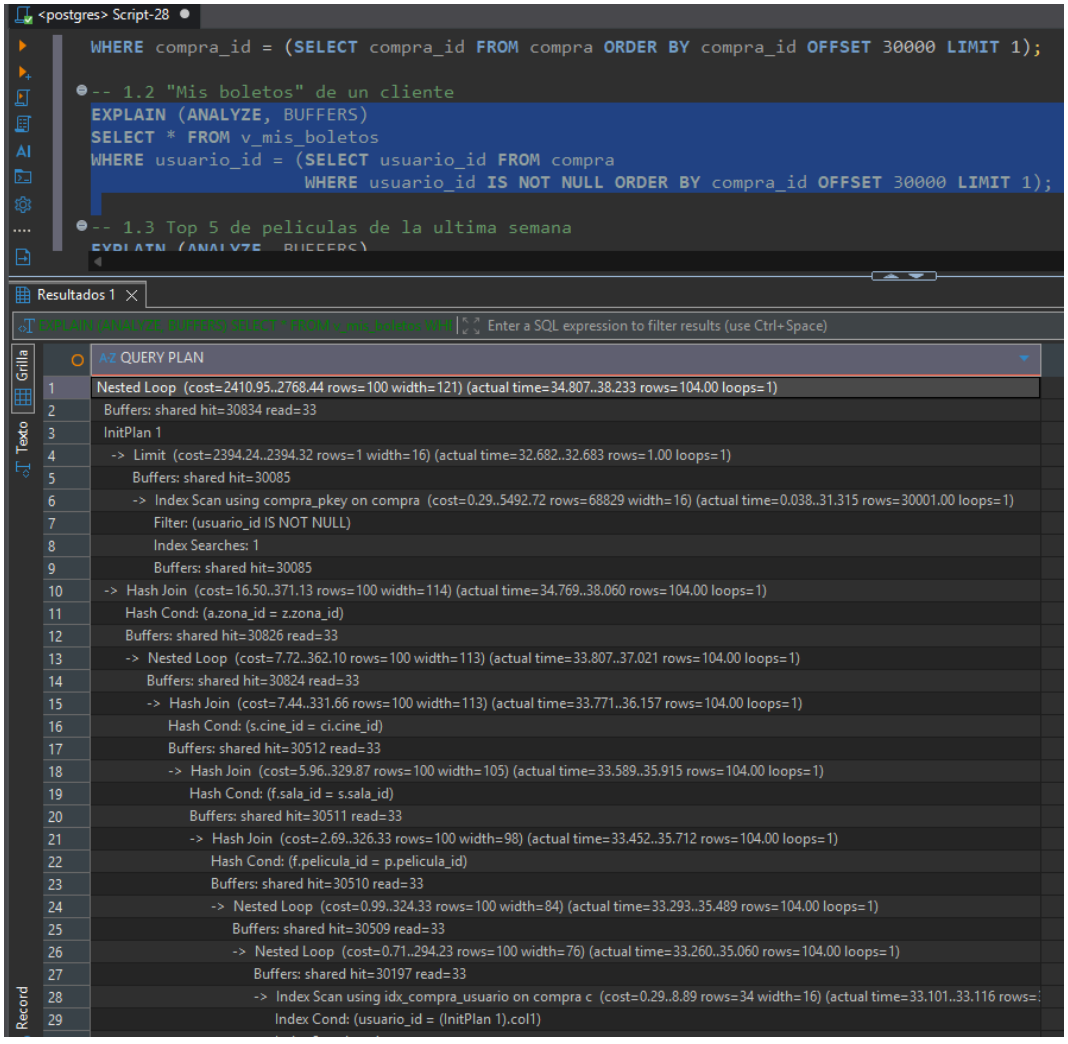


Figura 27
Consulta 1.3 después del índice: top 5 de películas de la semana

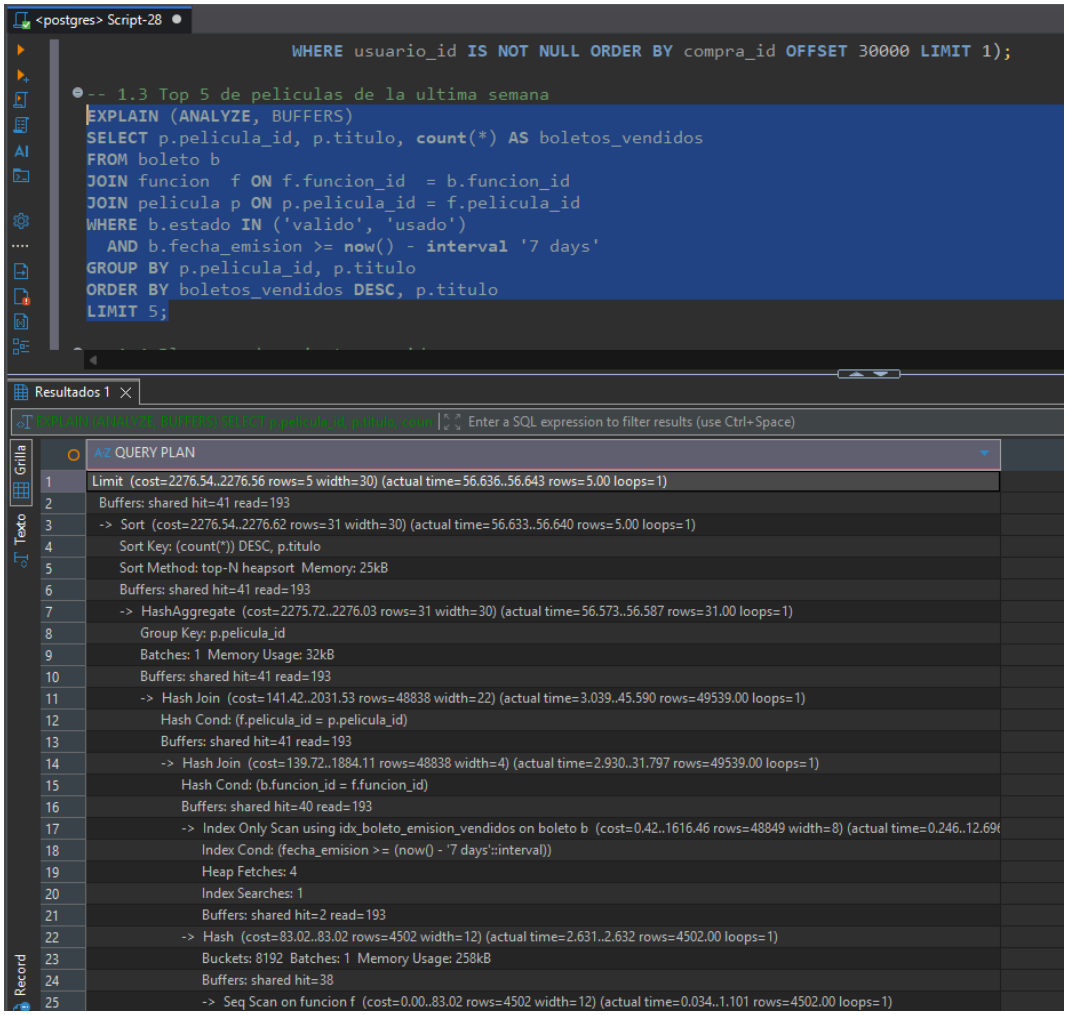


Figura 28
Consulta 1.4 después del índice: bloqueos de asiento vencidos

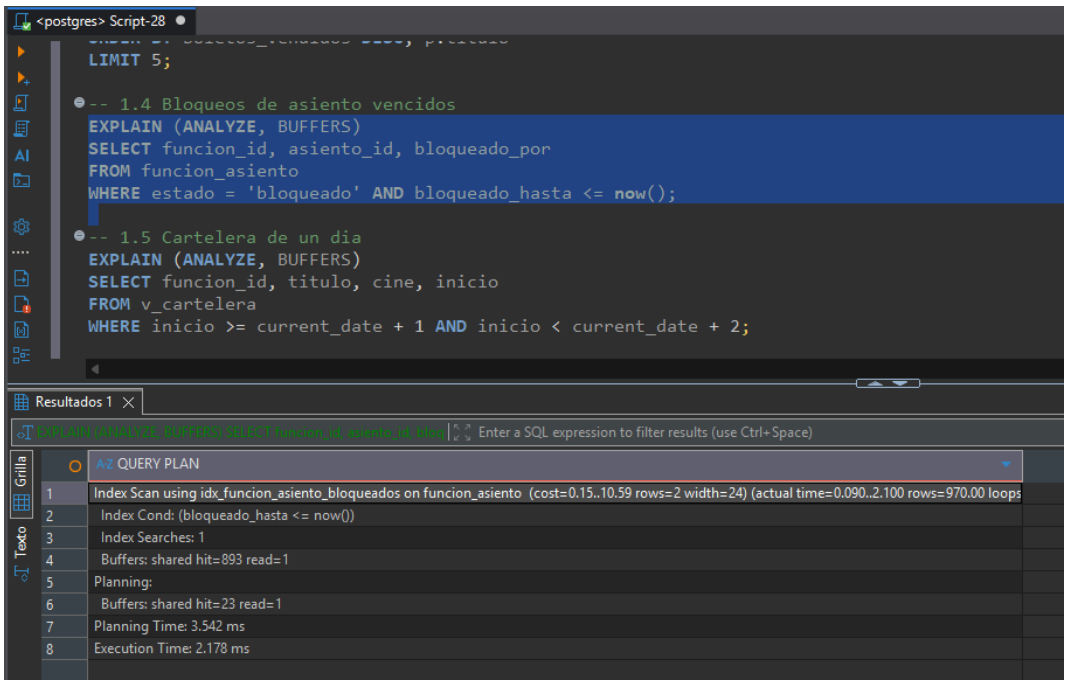


Figura 29
Consulta 1.5 después del índice: cartelera de un día

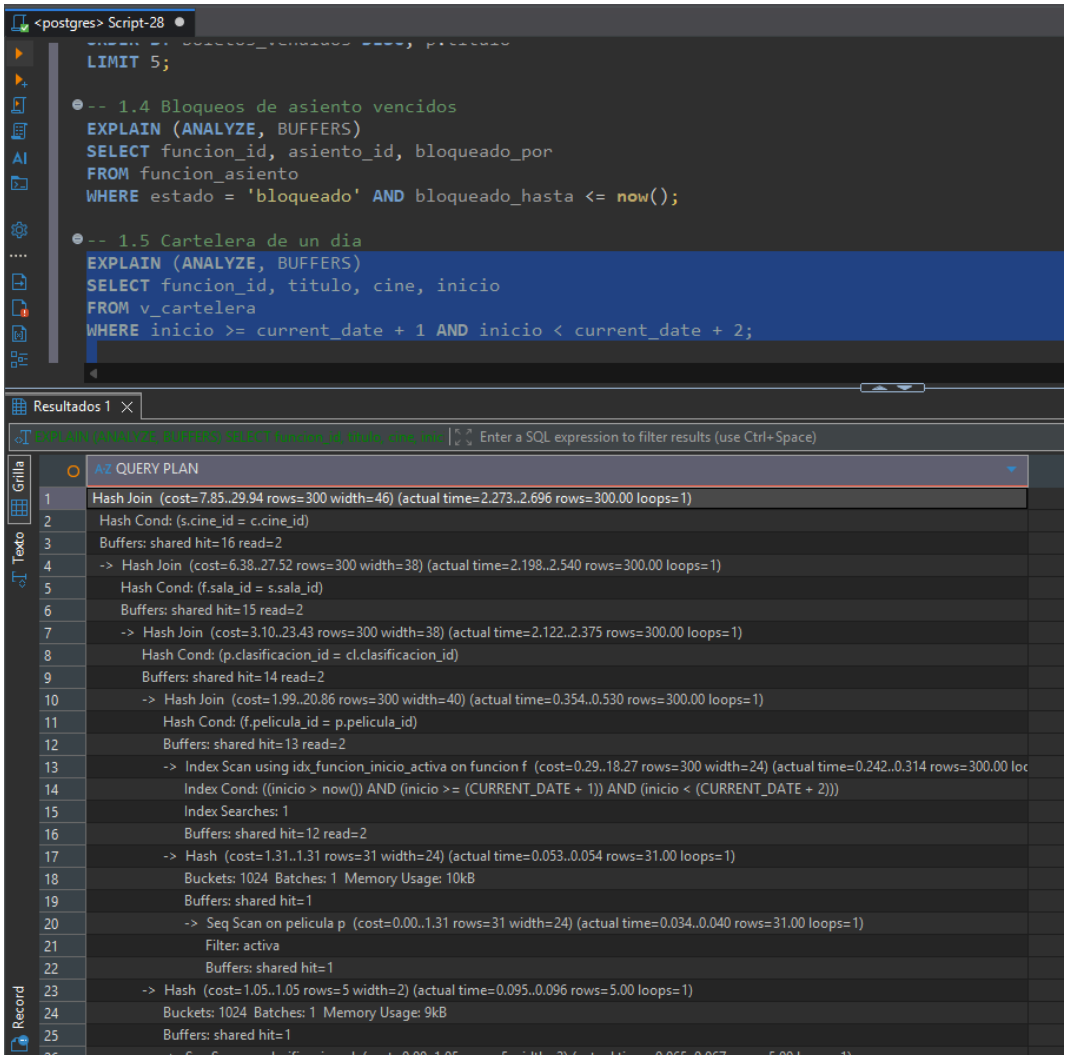


Figura 30

Consulta crítica: mapa de asientos disponibles de una función

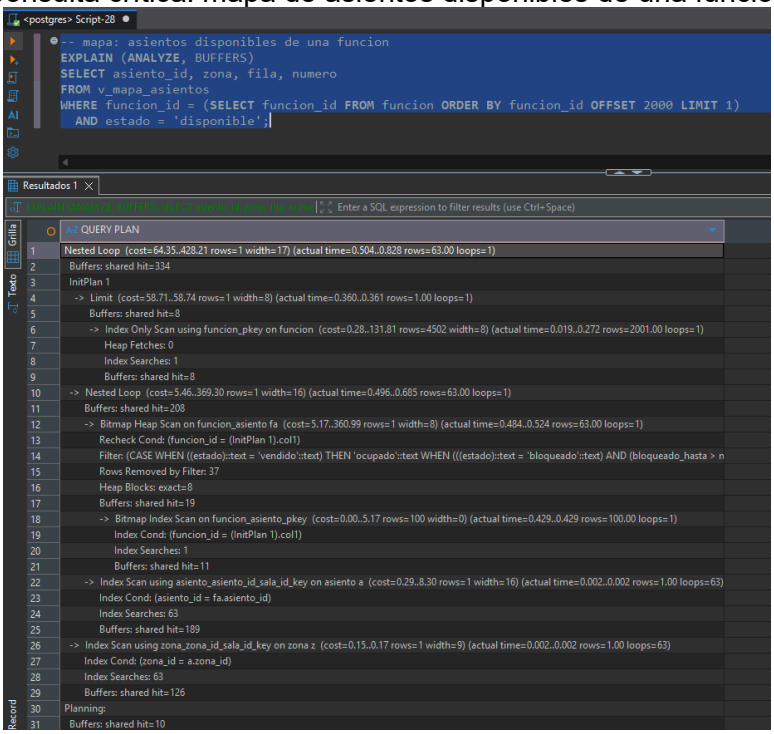


Figura 31
Consulta crítica: validación de un QR en la puerta

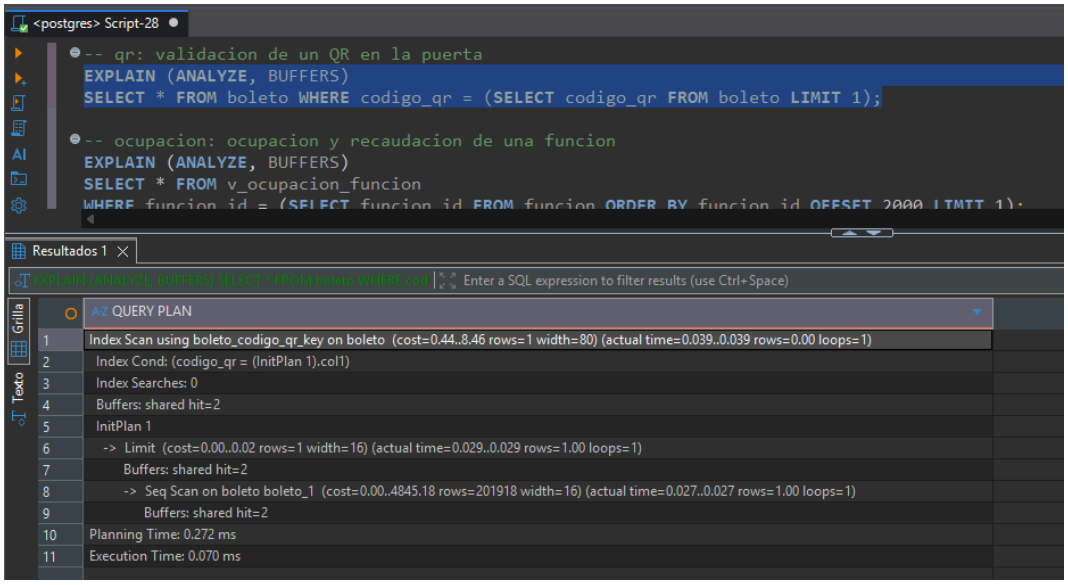


Figura 32
Consulta crítica: ocupación y recaudación de una función

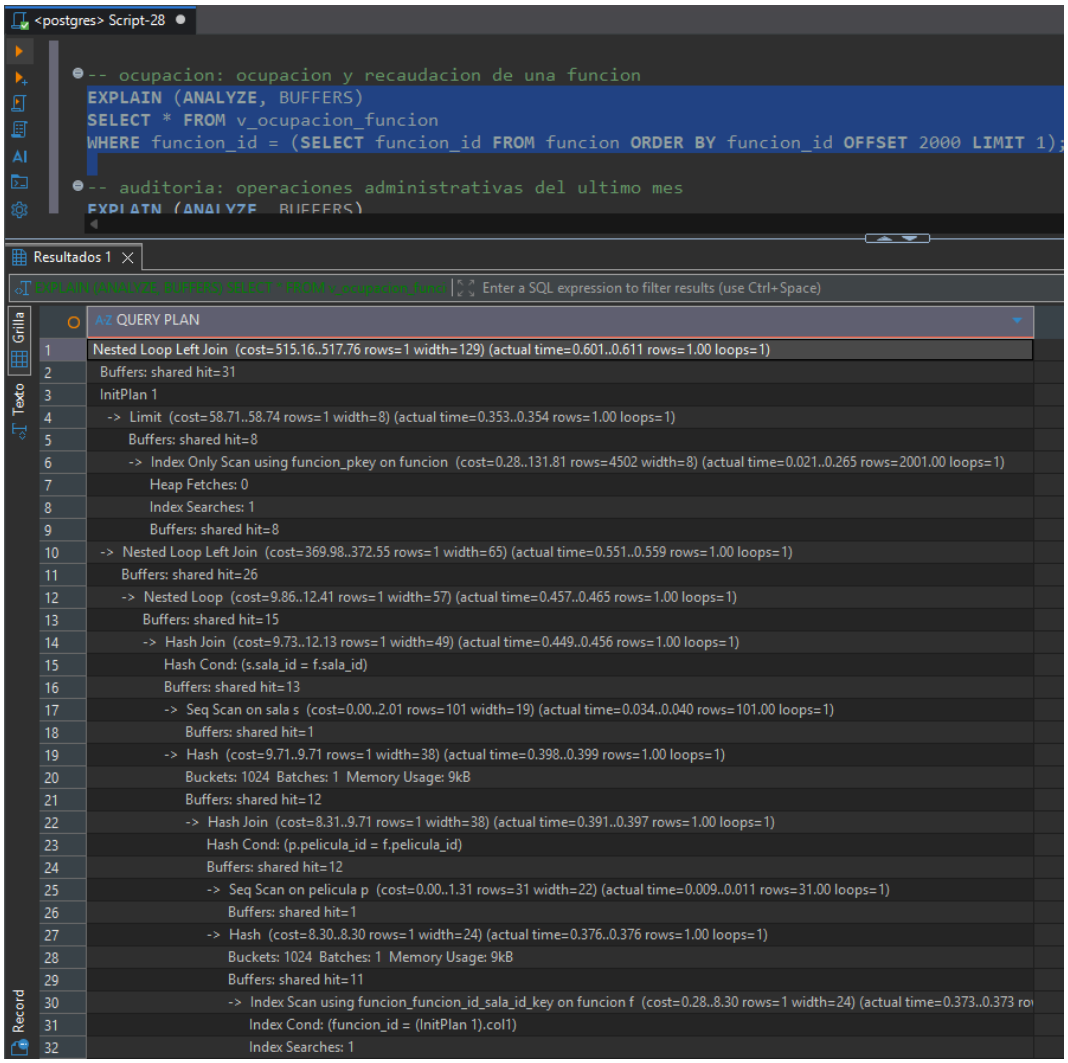
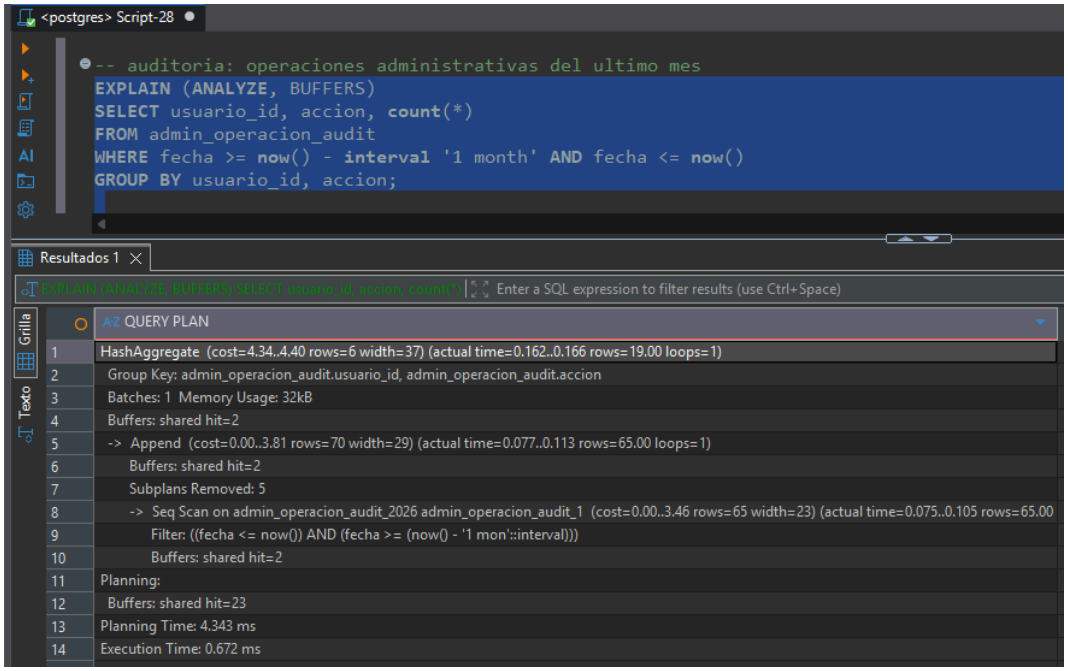


Figura 33
Consulta crítica: auditoría del último mes con poda de particiones



6. Operación y mantenimiento

6.1 Respaldos. Proponemos un respaldo continuo de los registros de transacciones, para poder restaurar a cualquier momento de los últimos 35 días, una copia completa diaria automática, una copia semanal replicada en otra región y una copia mensual en almacenamiento de archivo que no se pueda modificar, guardada 5 años. Cada trimestre se prueba restaurar una copia para confirmar que sirve.

6.2 RPO de 5 segundos o menos. La base de datos replica de forma sincrónica entre zonas, así que si cae una zona no se pierden datos, y de forma asíncrona hacia otra región. Esto importa porque perder datos significa ventas sin registrar, doble venta y problemas legales.

6.3 RTO menor a 5 minutos. La base de datos y el tráfico cambian automáticamente a la réplica, sin que nadie tenga que intervenir, y las réplicas ya están encendidas en otra zona. Cada minuto caído durante un estreno mundial es una pérdida enorme.

6.4 SLA de 99.99%. Equivale a máximo 52.6 minutos de caída al año, o 4.3 minutos al mes. Lo respaldamos con servicios administrados con alta disponibilidad entre zonas y con redundancia en cada capa. El SLA total es el producto de los SLA de las partes que van en serie, por eso ninguna capa depende de un solo componente.

6.5 Recuperación ante desastres. Usamos un esquema activo-pasivo en dos regiones, con pasos documentados para cada tipo de falla y simulacros de caída de zona y de región dos veces al año.

6.6 Privacidad de datos. Guardamos solo los datos personales necesarios, cifrados y con acceso por rol. Hay un cruce entre privacidad y retención: la norma fiscal pide guardar 5 años, pero las leyes de privacidad permiten pedir el borrado de datos personales. Lo resolvemos guardando los registros financieros los 5 años y anonimizando los datos personales que no sean obligatorios.

7. Interfaz gráfica

La interfaz web es una aplicación Flask (carpeta gui/ del repositorio) que se conecta a PostgreSQL con psycopg2. No tiene lógica de negocio propia: cada acción llama a una función o a una vista de la base de datos (por ejemplo fn_crear_compra, fn_pagar_compra, fn_validar_entrada, v_cartelera y v_mapa_asientos). La aplicación entra a la base con el usuario app_imax (script 11_app_gui.sql), que no tiene permisos propios sobre las tablas, y en cada petición ejecuta SET LOCAL ROLE con el rol de la persona que inició sesión, de modo que los privilegios de la sección 4.4 se aplican también aquí. Las contraseñas se verifican en la base con bcrypt (fn_login_usuario, script 12_login_seguro.sql).

7.1 Cómo ejecutarla

Primero se corren, como postgres, los scripts 01 a 07b y 09 (datos de prueba), y después 11_app_gui.sql y 12_login_seguro.sql. Luego, en la carpeta gui, se ejecutan pip install -r requirements.txt y python app.py, y se abre http://127.0.0.1:5000. La aplicación busca por defecto la base imax en localhost; si la base tiene otro nombre o la conexión es distinta, se indica con las variables de entorno IMAX_DB_NAME, IMAX_DB_HOST, IMAX_DB_PORT, IMAX_DB_USER e IMAX_DB_PASSWORD. Los usuarios de prueba tienen como contraseña el nombre del rol seguido de 123:

Rol	Correo	Contraseña
Cliente	cliente1@demo.imax	cliente123
Taquillero	taquillero1@demo.imax	taquillero123
Operador de sala	operador_sala1@demo.imax	operador_sala123
Contador	contador1@demo.imax	contador123
Administrador	administrador1@demo.imax	administrador123

7.2 Cliente

El cliente filtra la cartelera por película, género, ciudad, cine, clasificación y fecha (Figura 34). Al elegir función y asientos, la aplicación llama a fn_bloquear_asiento y fn_crear_compra y muestra el detalle de la compra con un temporizador de 2 minutos para pagar (RN-05), como en la Figura 35: los dos boletos de platea de Película Demo 01 suman 10,000, la promoción 2x1 descuenta 5,000 y el total es 5,000. Los campos de la tarjeta son una simulación: se validan solo en el navegador y no se envían ni se guardan (RN-09).

Figura 34

Cartelera con filtros (sesión de Cliente Demo 1)

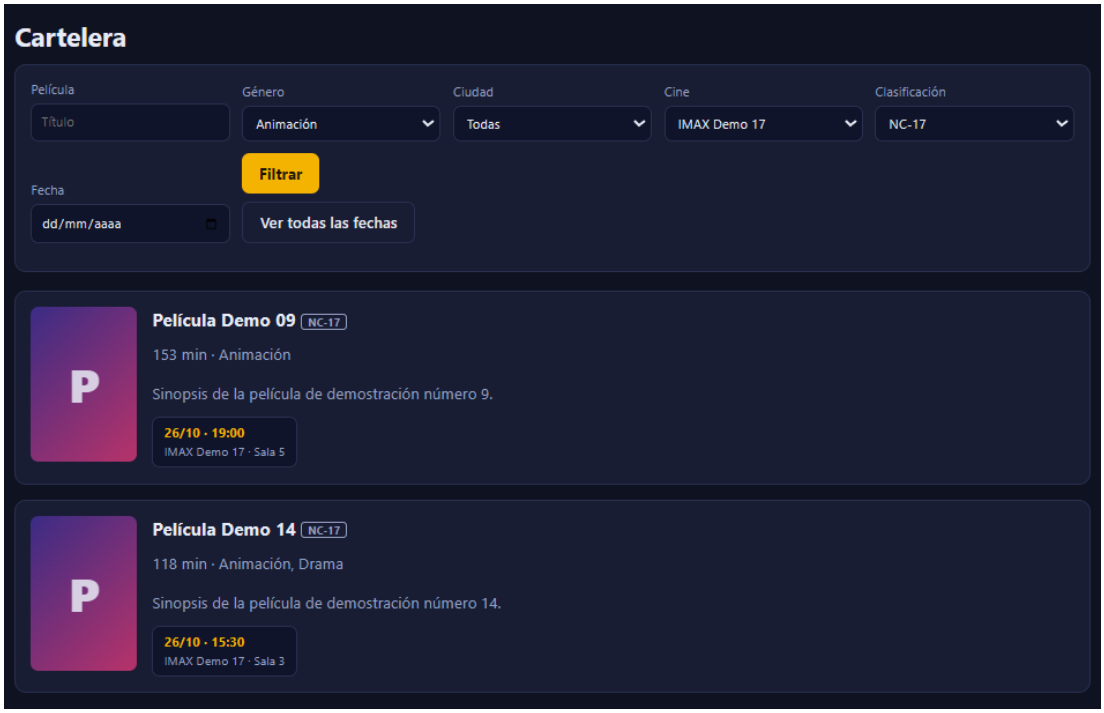


Figura 35
Detalle de la compra y pago simulado

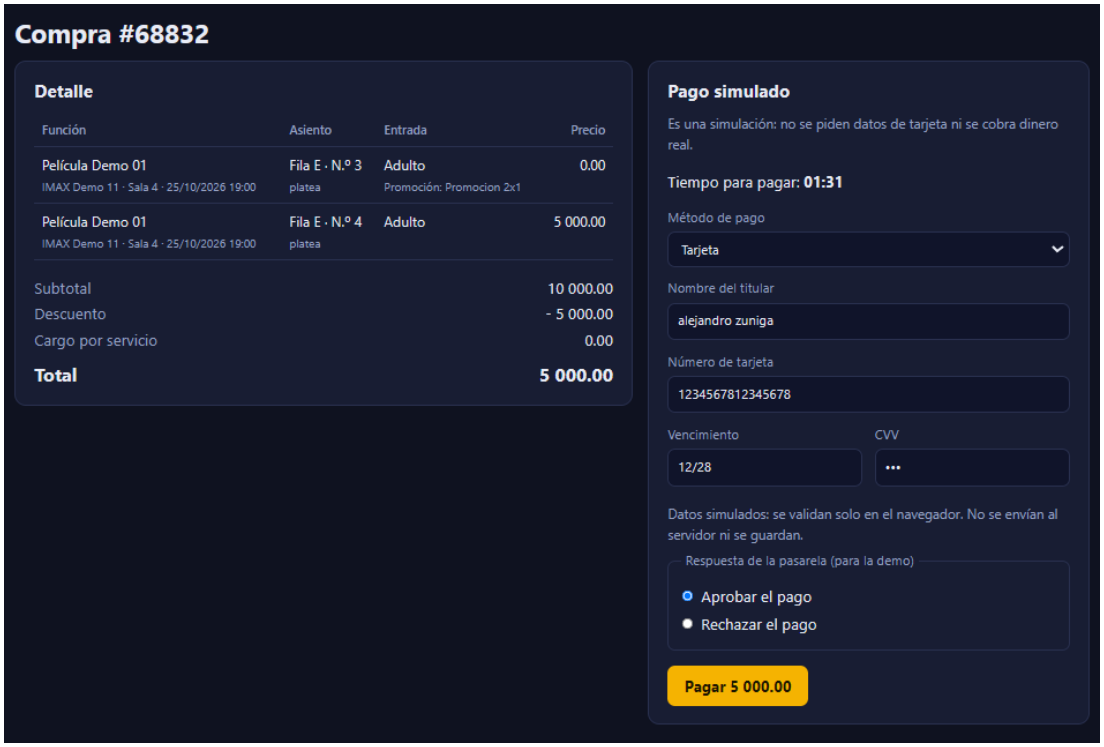


Figura 36
Entradas con código QR después de un pago aprobado



Figura 37
Compra rechazada: la compra se anula y los asientos se liberan



7.3 Taquillero y operador de sala

El taquillero ve el resumen de sus ventas en taquilla por día y método de pago (Figura 38). El operador de sala escribe o escanea el código QR e indica la función y la puerta; fn_validar_entrada responde y deja el intento en acceso_puerta_log. En la Figura 39 se validó el QR de un boleto de la compra n.º 68832 contra una función distinta de la del boleto y la base lo rechazó (RN-15). Las otras respuestas de la función (código inexistente, entrada ya utilizada y fuera de la tolerancia de horario) están en el script 04_checkin_reembolsos.sql.

Figura 38
Corte de caja del taquillero



Figura 39
Validación de una entrada en la puerta: rechazada por ser de otra función



7.4 Administrador

El panel de administración, que solo abre el rol administrador, tiene seis pestañas: Resumen, Películas, Funciones, Precios y promociones, Salas y Operaciones y bitácora.

Películas y salas. El formulario de película pide título, sinopsis, duración, clasificación, póster opcional y géneros (Figura 40); al guardar, la película queda lista para programarse en una función (Figura 41). Una sala nueva se define con filas y asientos por fila; en la Figura 42, la sala nueva tiene 400 asientos en una zona.

Figura 40
Alta de cartelera: formulario de nueva película

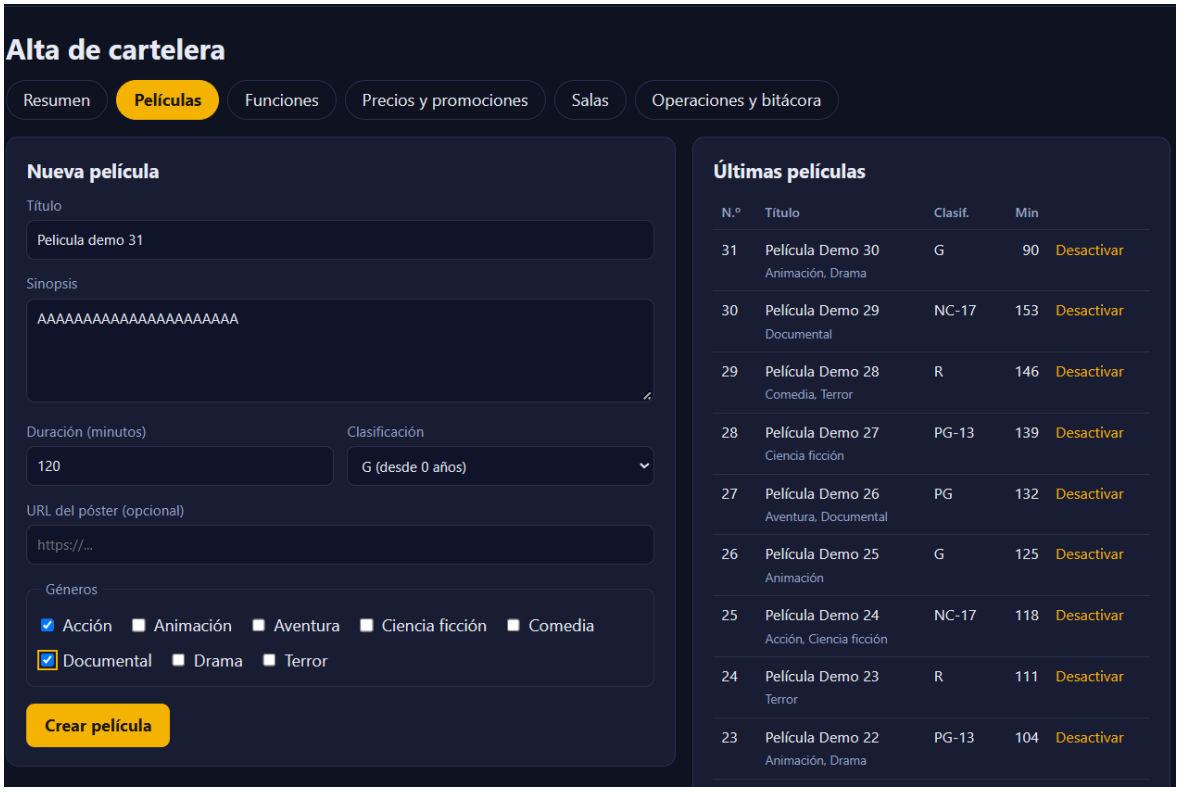


Figura 41
Película creada

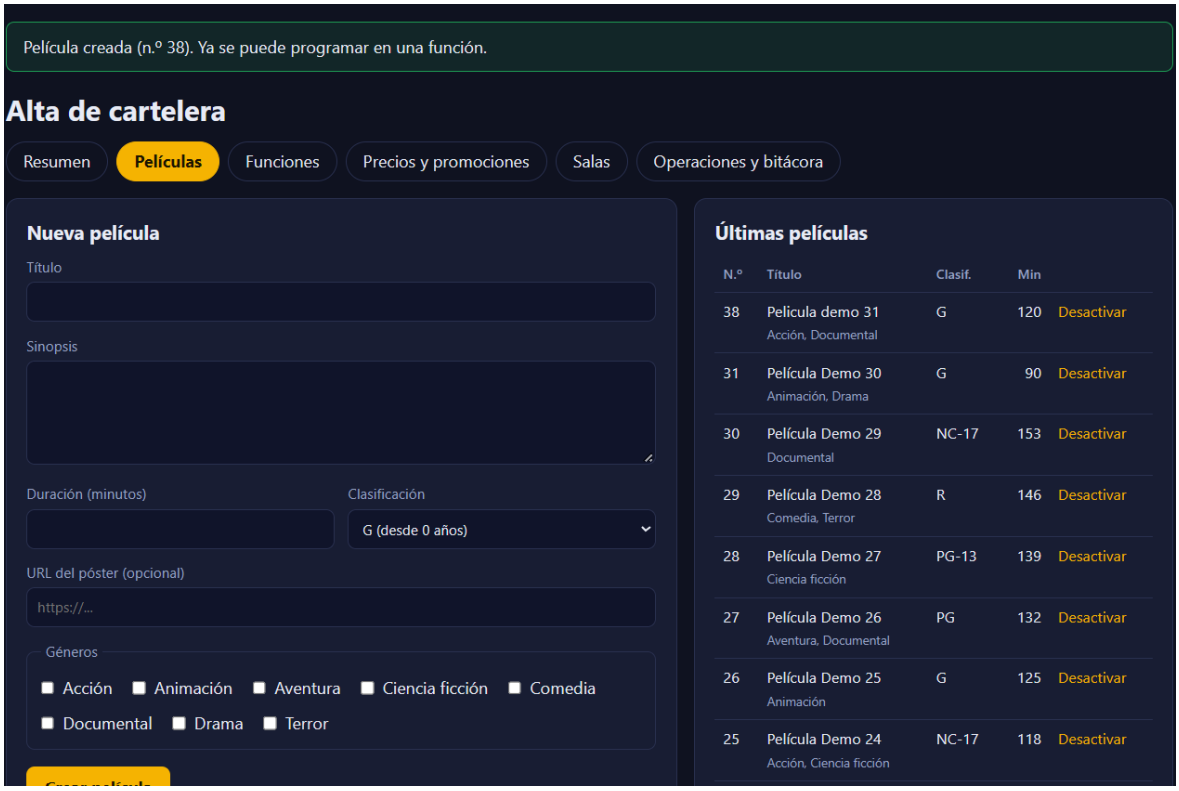
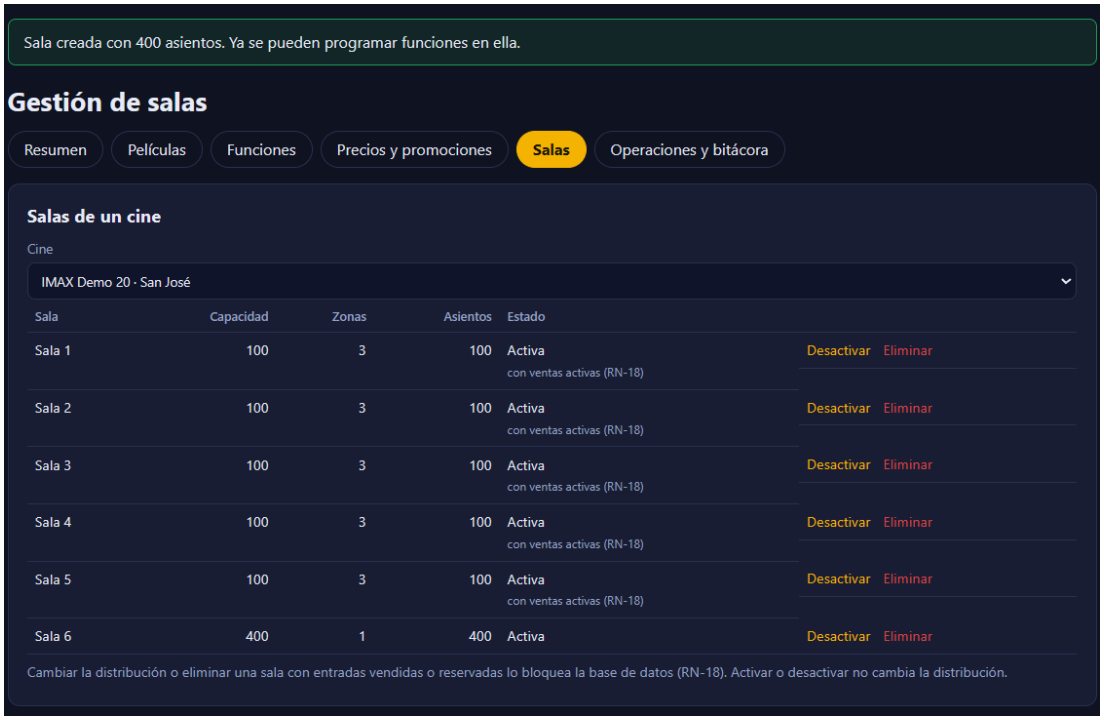


Figura 42
Sala creada con 400 asientos



Funciones y horarios. Para programar una función se elige película, sala, hora de inicio y minutos de limpieza, y se llena la matriz de precios por zona y tipo de entrada (Figura 43). La aplicación calcula la hora de fin y, al confirmar, los asientos quedan disponibles para la venta (Figura 44). Si el horario se traslapa con otra función de la misma sala, la restricción EXCLUDE de la base lo rechaza (Figura 45; sección 5.1).

Figura 43
Formulario para programar una función con su matriz de precios



Figura 44
Función creada con su matriz de precios

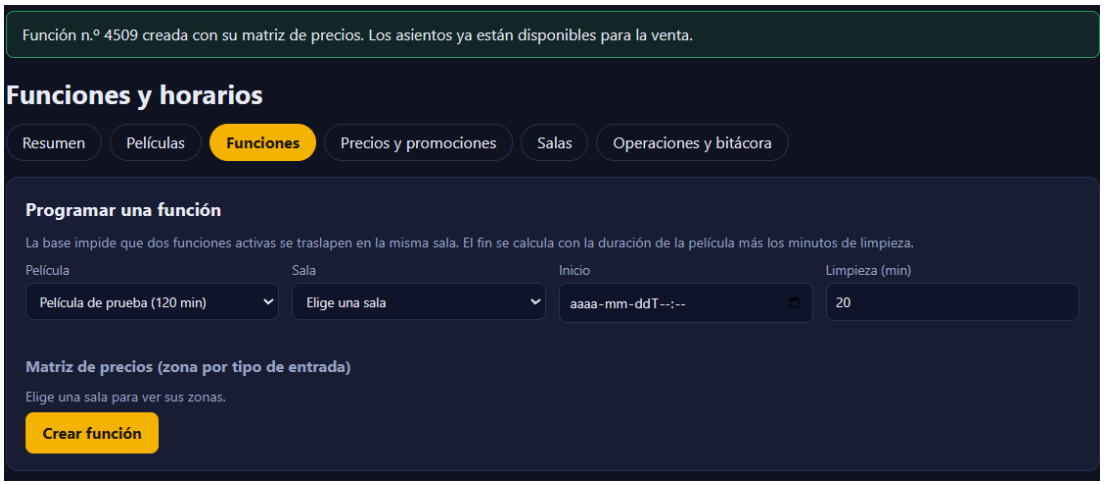
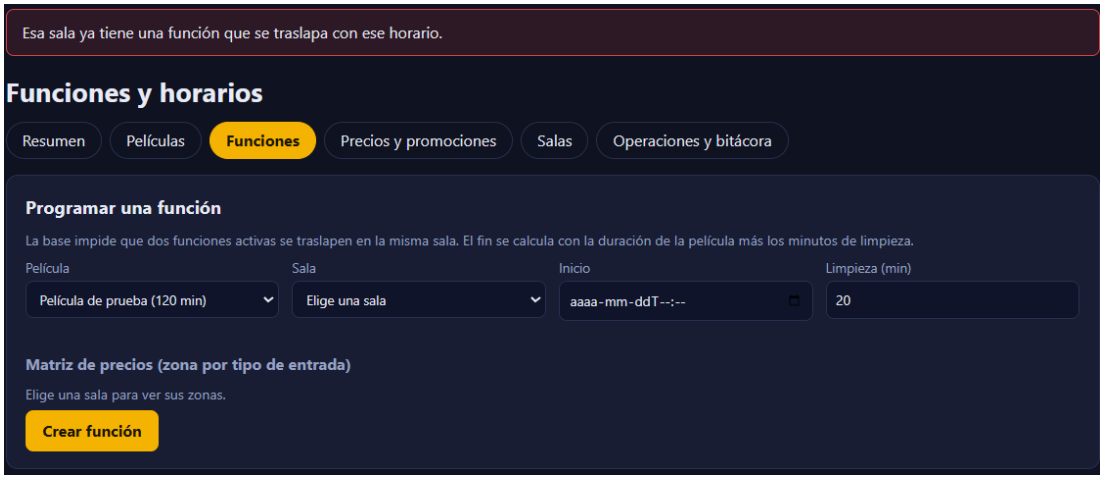
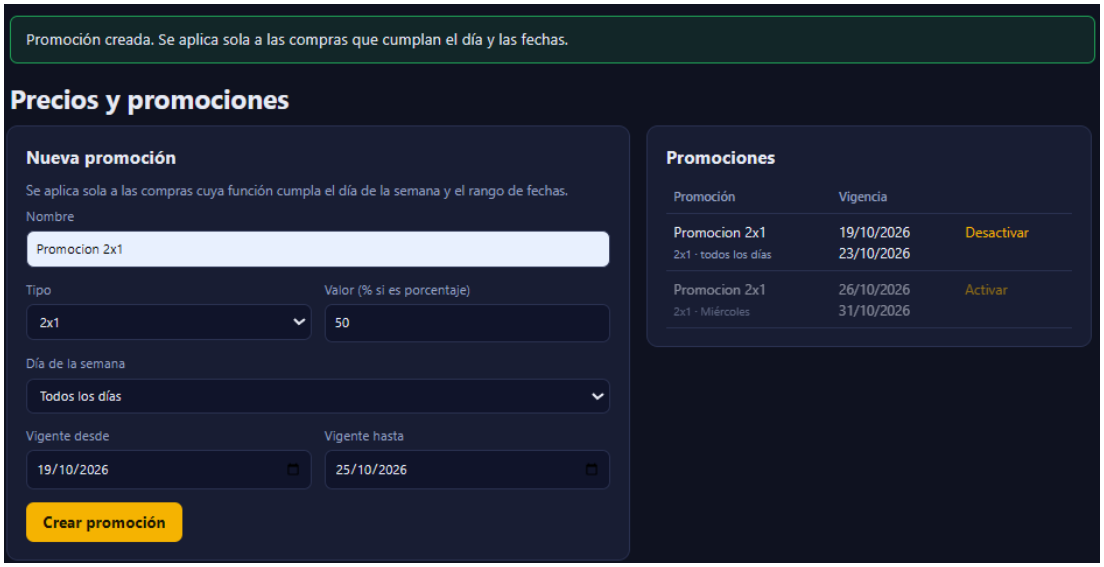


Figura 45
Rechazo de una función que se traslapa con otra en la misma sala



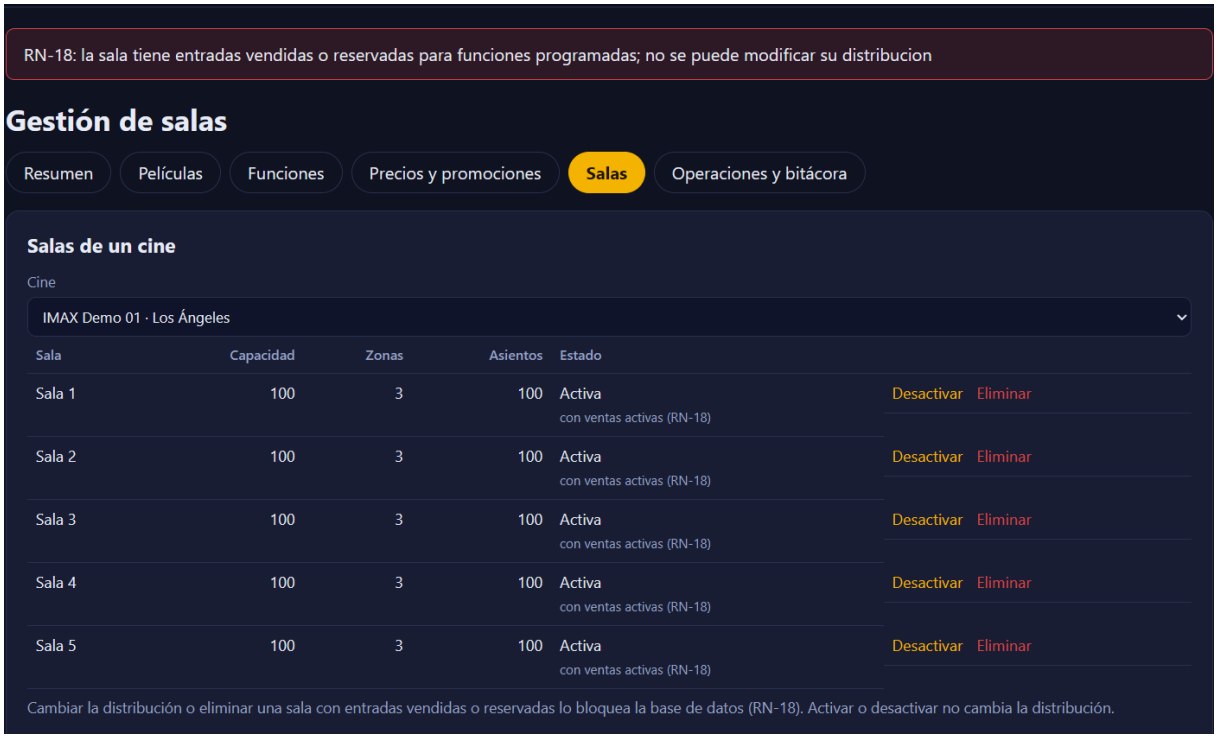
Precios y promociones. El administrador define promociones (2x1 o porcentaje) con día de la semana y vigencia; se aplican solas en las compras que cumplen las condiciones (Figura 46). Cambiar una tarifa exige un motivo y queda en la bitácora.

Figura 46
Promociones: confirmación, formulario y lista de promociones



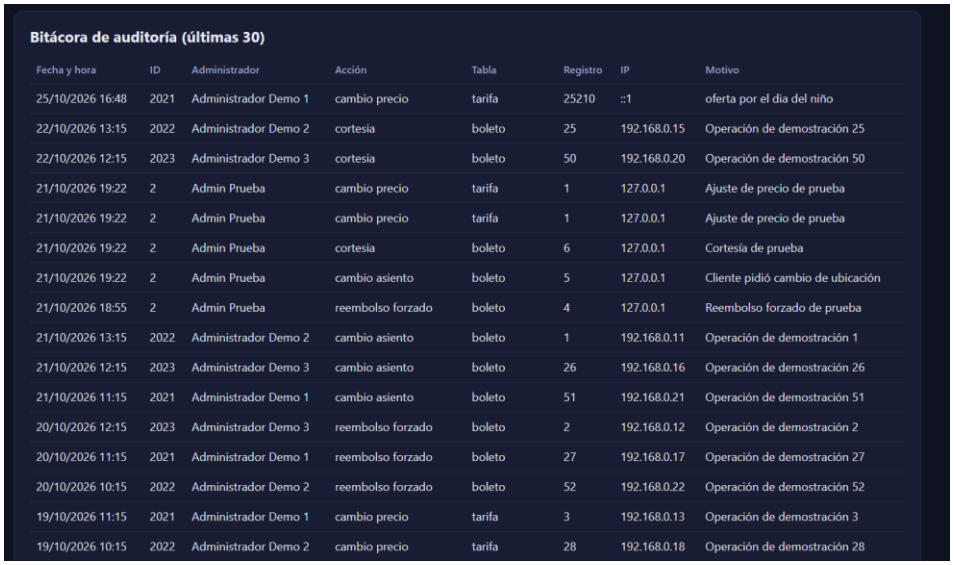
Salas con ventas (RN-18). Si una sala tiene entradas vendidas o reservadas para funciones programadas, la base rechaza los cambios en su distribución y la aplicación muestra el mensaje del trigger (Figura 47). Esas salas llevan la etiqueta «con ventas activas (RN-18)».

Figura 47
Rechazo de un cambio en una sala con ventas (RN-18)



Operaciones manuales y bitácora. Los reembolsos forzados, cambios de asiento, cortesías y cambios de precio o de configuración exigen un motivo y dejan una fila en la bitácora con fecha, administrador, acción, tabla, registro, IP y motivo (RN-20). La Figura 48 muestra las operaciones más recientes.

Figura 48
Bitácora de auditoría: operaciones más recientes



Bitácora de auditoría (últimas 30)

Fecha y hora	ID	Administrador	Acción	Tabla	Registro	IP	Motivo
25/10/2026 16:48	2021	Administrador Demo 1	cambio precio	tarifa	25210	::1	oferta por el dia del niño
22/10/2026 13:15	2022	Administrador Demo 2	cortesia	boleto	25	192.168.0.15	Operación de demostración 25
22/10/2026 12:15	2023	Administrador Demo 3	cortesia	boleto	50	192.168.0.20	Operación de demostración 50
21/10/2026 19:22	2	Admin Prueba	cambio precio	tarifa	1	127.0.0.1	Ajuste de precio de prueba
21/10/2026 19:22	2	Admin Prueba	cambio precio	tarifa	1	127.0.0.1	Ajuste de precio de prueba
21/10/2026 19:22	2	Admin Prueba	cortesia	boleto	6	127.0.0.1	Cortesía de prueba
21/10/2026 19:22	2	Admin Prueba	cambio asiento	boleto	5	127.0.0.1	Cliente pidió cambio de ubicación
21/10/2026 18:55	2	Admin Prueba	reembolso forzado	boleto	4	127.0.0.1	Reembolso forzado de prueba
21/10/2026 13:15	2022	Administrador Demo 2	cambio asiento	boleto	1	192.168.0.11	Operación de demostración 1
21/10/2026 12:15	2023	Administrador Demo 3	cambio asiento	boleto	26	192.168.0.16	Operación de demostración 26
21/10/2026 11:15	2021	Administrador Demo 1	cambio asiento	boleto	51	192.168.0.21	Operación de demostración 51
20/10/2026 12:15	2023	Administrador Demo 3	reembolso forzado	boleto	2	192.168.0.12	Operación de demostración 2
20/10/2026 11:15	2021	Administrador Demo 1	reembolso forzado	boleto	27	192.168.0.17	Operación de demostración 27
20/10/2026 10:15	2022	Administrador Demo 2	reembolso forzado	boleto	52	192.168.0.22	Operación de demostración 52
19/10/2026 11:15	2021	Administrador Demo 1	cambio precio	tarifa	3	192.168.0.13	Operación de demostración 3
19/10/2026 10:15	2022	Administrador Demo 2	cambio precio	tarifa	28	192.168.0.18	Operación de demostración 28

La aplicación también incluye el mapa de asientos en vivo con temporizador de bloqueo, un dashboard que se actualiza cada 15 segundos para el contador y el administrador, y las pantallas de reembolso forzado, cambio de asiento y cortesía; su código está en gui/ (app.py, admin.py y static/mapa.js).

7.5 Alcance y limitaciones

- El pago es una simulación: no hay pasarela real y la respuesta aprobada o rechazada se elige en pantalla para la demostración.
- El usuario de la base (app_imax), las contraseñas de los usuarios de prueba y la clave de sesión de Flask son valores de demostración; en producción deben cambiarse y leerse de variables de entorno.
- La aplicación se ejecuta con python app.py (servidor de desarrollo de Flask). La prueba de concurrencia de la sección 5.3 se hizo directamente contra la base de datos con 30 conexiones, no a través de la interfaz.

8. Repositorio y evidencias

Todo el código y las evidencias están en un repositorio público de GitHub: https://github.com/azuniga7/Proyecto-2--IMAX-licitaci-n

Estructura del repositorio

Carpeta o archivo	Contenido
sql/	Scripts de PostgreSQL, numerados en el orden en que se ejecutan (01 a 12, más 07b).
gui/	Interfaz web en Flask (app.py, admin.py, plantillas, estilos y mapa.js) y su guía de ejecución.
py/	prueba_doble_venta.py y resultado_concurrencia.txt, la salida de la prueba de concurrencia.
Documentación/	Este documento.
Leer.txt	Descripción general del proyecto y cómo ejecutarlo.

Scripts de la carpeta sql/

Script	Qué contiene
01_ddl.sql	Esquema: 25 tablas, restricciones, índices únicos, bitácoras particionadas y datos base.
02_funciones_triggers.sql	Bloqueo y liberación de asientos y creación automática del estado de cada asiento por función.
03_compra_pago.sql	Compra y pago simulado (fn_crear_compra, fn_pagar_compra, fn_anular_compra_pendiente).
04_checkin_reembolsos.sql	Validación del QR en la puerta y reembolsos.
05_auditoria_salas.sql	Protección de salas con ventas (RN-18), bitácoras inmutables y operaciones manuales del administrador.

Script	Qué contiene
06_vistas.sql	Vistas de cartelera, mapa de asientos, mis boletos, corte de caja, ocupación y ventas diarias.
07_roles.sql y 07b_ajustes_rbac.sql	Roles y privilegios mínimos (RN-17) y sus ajustes.
08_consultas_analiticas.sql	Consultas analíticas de la gerencia (sección 5.4).
09_datos_prueba.sql	Datos de prueba masivos, unos 202 mil boletos.
10_indices_explain.sql	Índices y comparación de EXPLAIN ANALYZE antes y después (sección 5.5).
11_rbac_demo.sql	Demostración de los privilegios por rol (sección 4.4).
11_app_gui.sql y 12_login_seguro.sql	Usuario de la aplicación, vistas de la compra y login con bcrypt (sección 7).

Cómo reproducir los resultados

Base de datos. Crear la base y ejecutar como postgres los scripts 01 a 07b y 09 en orden numérico. Los scripts 08, 10 y 11_rbac_demo se corren cuando se quieren reproducir las consultas analíticas, las mediciones de EXPLAIN y la prueba de roles.

Prueba de concurrencia. Ajustar la contraseña de postgres en la variable CONFIG de py/prueba_doble_venta.py y ejecutar python prueba_doble_venta.py; el resultado queda en resultado_concurrencia.txt.

Interfaz. Ejecutar 11_app_gui.sql y 12_login_seguro.sql, y seguir los pasos de la sección 7.1.

9. Declaración de IA:

Se utilizó IA como herramienta de ayuda, mejora constante y optimización del proyecto. Se utilizó para aclarar dudas, mejora de redacción, corrección de errores, ayuda en la búsqueda de los mismos en lo que es la base de datos. Se utilizo como herramienta de búsqueda de información, optimización de redacción, herramienta comparativa y como herramienta de búsqueda de errores para la entrega final en lo que es la documentación. Se utilizó de forma recurrente y completa en lo que es la app, no nos han enseñado ni tenemos los conocimientos completos para elaborar una app o una interfaz capaz de conectarse ni funcionar de manera autónoma, por lo que decidimos apoyarnos de gran manera en la IA, se utilizó Claude como herramienta constructiva y de apoyo para la app.

10. Referencias

Amazon Web Services. (s. f.-a). *Amazon Aurora pricing*. Recuperado el 20 de octubre de 2026, de https://aws.amazon.com/rds/aurora/pricing

Amazon Web Services. (s. f.-b). *AWS WAF pricing*. Recuperado el 20 de octubre de 2026, de https://aws.amazon.com/waf/pricing/

Bytebase. (s. f.). *Azure Database for PostgreSQL & MySQL pricing*. Recuperado el 20 de octubre de 2026, de https://www.bytebase.com/dbcost/azure-flexible-server-pricing/

Cloudian. (s. f.). *Understanding AWS S3 storage pricing: A comprehensive guide*. Recuperado el 20 de octubre de 2026, de https://cloudian.com/blog/5-components-of-aws-s3-storage-pricing/

CloudZero. (2026, 22 de julio). *Azure Blob Storage pricing*. https://cloudzero.com/blog/azure-blob-storage-pricing

Google. (s. f.-a). *Files you can store in Google Drive*. Ayuda de Google Drive. Recuperado el 20 de octubre de 2026, de https://support.google.com/drive/answer/37603

Google. (s. f.-b). *Usage limits*. Google for Developers. Recuperado el 20 de octubre de 2026, de https://developers.google.com/sheets/api/limits

Google Workspace Updates. (2026, 10 de septiembre). *Doubled cell limits in Google Sheets now generally available*. Workspace Updates Blog. https://workspaceupdates.googleblog.com/2026/09/doubled-cell-limits-in-google-sheets-now-generally-available.html

Gray, L. K. (2017, 2 de noviembre). *FAQ: Can CVC be stored for card on file or recurring transactions?* PCI Security Standards Council Blog. https://blog.pcisecuritystandards.org/faq-can-cvc-be-stored-for-card-on-file-or-recurring-transactions

Microsoft. (s. f.). *Azure Front Door pricing*. Azure. Recuperado el 20 de octubre de 2026, de https://azure.microsoft.com/en-us/pricing/details/frontdoor/

Microsoft. (2026). *Access specifications*. Soporte técnico de Microsoft. Recuperado el 20 de octubre de 2026, de https://support.microsoft.com/en-us/access/access-specifications

SQLite. (2025, 31 de mayo). *Appropriate uses for SQLite*. https://www.sqlite.org/whentouse.html